

Розробка програмного забезпечення із доповненою реальністю з використанням JavaScript

Мета – навчитися створювати програми із доповненою реальністю, орієнтовані на традиційні та мобільні веб-браузери, за допомогою бібліотек A-Frame, Three.js, AR.js та ін.

Пререквізити – успішне опанування курсу «Web-програмування».

Обладнання – стаціонарний або мобільний комп'ютерний засіб, обладнаний зовнішньою або вбудованою камерою.

Програмне забезпечення – веб-браузер із підтримкою WebGL, текстовий редактор.

Анотація. Ідея доповненої реальності (Augmented Reality – AR) полягає у комбінуванні комп'ютерно-генерованих моделей з об'єктами реального світу. AR.js надає набір засобів для ефективної розробки AR-програм за допомогою веб-технологій, таких як JavaScript і HTML. Вступ містить основ AR та короткий огляд усіх бібліотек та засобів розробки. Робота розпочинається з налаштування базової структури AR-програми з використанням бібліотеки AR.js. Далі виконується налаштування камери для локалізації маркерів AR. Після цього виконується накладання 3D-контенту поверх маркерів за допомогою бібліотеки Three.js з використанням реалістичного освітлення та тіней до 3D-об'єктів з різними елементами керування, що надаються Three.js для навігації по 3D-сцені. Пропонується кілька проектів для закріплення набутих навичок створення AR-програм за допомогою A-Frame, Three.js, AR.js та інших бібліотек.

1 ТЕХНОЛОГІЇ ДОПОВНЕНОЇ РЕАЛЬНОСТІ

1.1. Вступ

Програмувати доповнену реальність – інноваційно (модно, цікаво, корисно та ін.) останні 60 років, і вибір JavaScript в якості мови програмування – виключно данина часу. Але вибір мови визначає й вибір засобів розробки, найбільш доцільними з яких на сьогодні є:

а) A-Frame та AR.js – ці API, фактично, унікальні засоби швидкого прототипування, і значна частина програми з їх використанням – це HTML-подібний код, який використовує JavaScript на сервері. A-Frame використовується для створення сцен, об'єктів, анімації та інших 3D-елементів у веб-браузері. AR.js надає можливість відслідковувати маркер і надає можливість сцені, сконструйованій за допомогою A-Frame, відображатися прямо на маркері;

б) Three.js та ARToolKit – своєрідний кістяк, який використовуються багато інших бібліотек мовою JavaScript. Three.js використовує WebGLRenderer, що надає можливість створення якісних 3D-сцени безпосередньо у браузері. На відміну від A-Frame, Three.js використовується в основному для створення веб-програм під управлінням Google Cardboard та вимагає явного використання JavaScript.

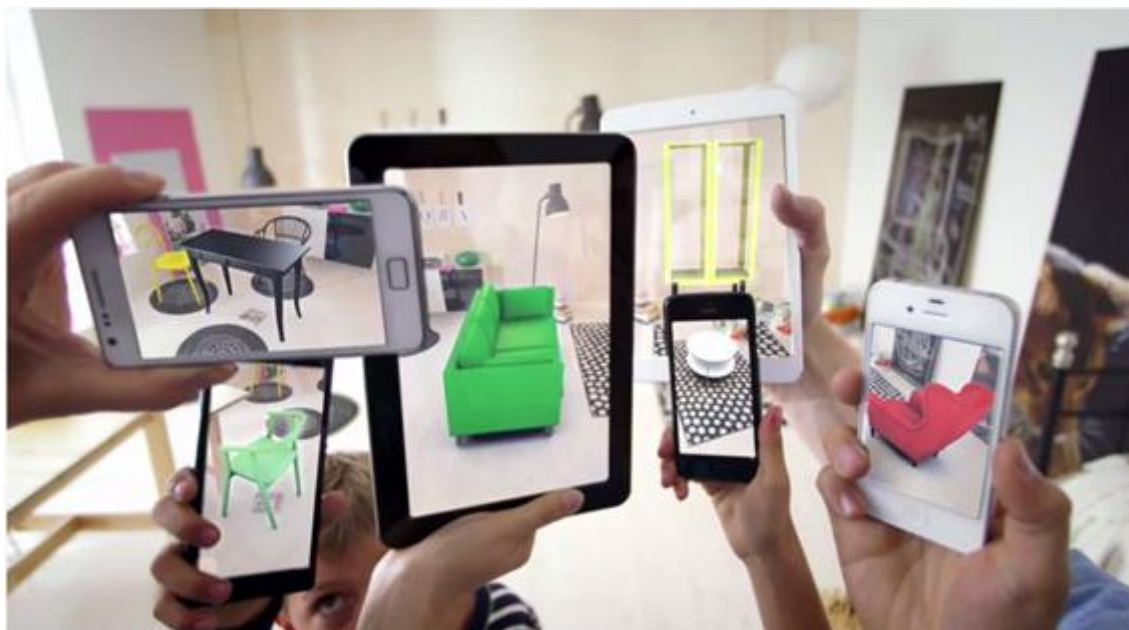
Програмні засоби із доповненою реальністю, розроблені із використанням JavaScript та WebGL, можуть бути розміщені в Інтернет на одному із хмарних сервісів, таких як Heroku.

1.2. Як працює доповнена реальність

Для початку роботи необхідно мати лише AR-сумісний браузер, такий як Firefox або Chrome, та текстовий редактор (типу mcedit або Sublime). Базові знання HTML, CSS та JavaScript є обов'язковими, а досвід роботи з веб-API та GitHub стануть у пригоді.

Надалі під AR будемо розуміти здатність пристрою, зокрема мобільного

пристрою або веб-браузера, відстежувати зображення або відображати 3D-об'єкт поверх цього зображення. Головна ідея AR полягає в тому, щоб відобразити комп'ютерну модель у реальному часі та реальному просторі з метою взаємодії між користувачем у реальному просторі та 3D-моделі у віртуальному.



AR може бути як маркерним, так й безмаркерним. У маркерній AR пристрій відстежує 2D-маркер: коли він знаходиться, на ньому фактично відображається 3D-об'єкт. У безмаркерному варіанті пристрій буде шукати плоску поверхню (стіл, підлогу тощо), і розташовуватимемо 3D-об'єкт на ній.

Використовуючи камеру пристрою, AR надає можливості відображення комп'ютерно згенерованих об'єктів в ігрових, маркетингових та інших програмах – наприклад, для розстановки меблів у вітальні або примірки одягу перед їх покупкою. Це дійсно велика можливість для бізнесу – показати, як виглядає продукція, перш ніж будь-який споживач дійсно її купує.

Для AR розробляються спеціальні пристрої, як правило, у вигляді шоломів та гарнітур, що надають можливість занурення користувача у модельне середовище.



1.3. Порівняння доповненої та віртуальної реальності

AR доповнює реальний світ 3D-моделями, якими можна керувати за допомогою мобільного пристрою в будь-якому місці. Віртуальна реальність (Virtual Reality – VR) занурює користувача у модельний світ, для чого, як правило, необхідні наголовні дисплеї (Head Mounted Devices – HMD).



Інтерактивність у програмах для AR і VR забезпечується дуже схоже. Так, наприклад, VR фактично використовує контролери, а у деяких випадках

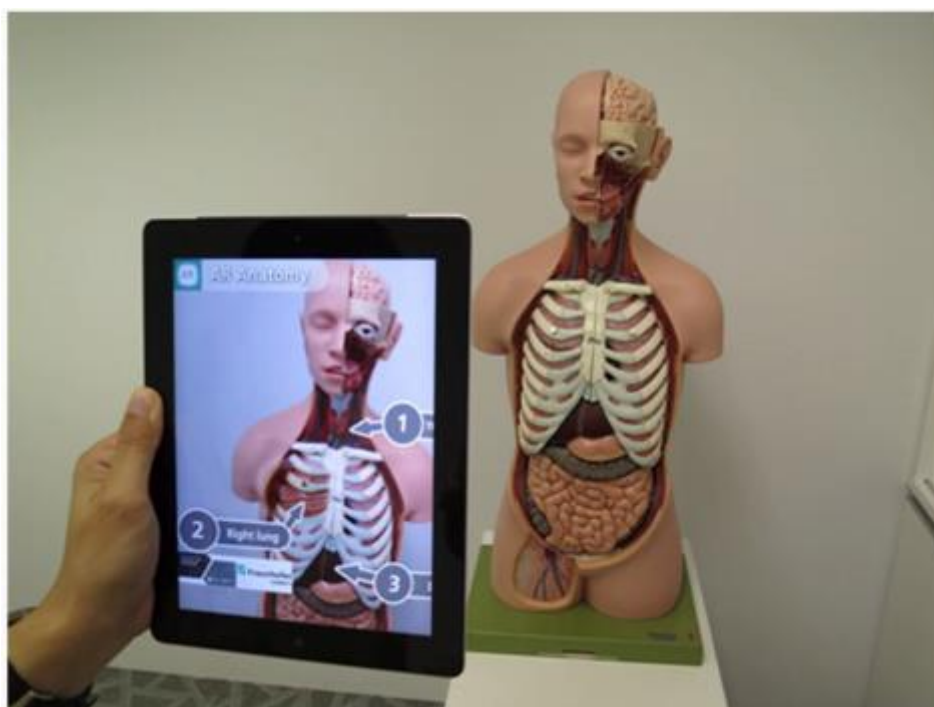
й відстеження рук, що надає змогу користувачеві взаємодіяти з 3D-об'єктами всередині сцени, у якій вони знаходяться.

До головних небезпек використання HMD для роботи у VR відносяться:

- напруження очей;
- запаморочення і головні болі після використання HMD.



На відміну від VR, AR не має таких значних ризиків для здоров'я. Тим не менш, викликає занепокоєння можливість користувачів залишатися зосередженими на тому, що вони роблять, під час використання AR – зокрема, з причин безпеки.



1.4. Пристрої доповненої реальності

Найбільш поширений тип пристроїв, готових для AR – смартфони та планшети з операційними системами iOS (версія 11 та вище під управлінням iPhone та iPad) та Android (версія 7 та вище).



Для веб-браузерів AR доступна, якщо у них реалізована підтримка WebRTC та WebGL – насамперед Google Chrome та Mozilla Firefox.

HoloLens є HMD-подібною гарнітурою для AR, що знаходиться у активній розробці. Цю гарнітуру часто порівнюють з AR-окулярами, такими як CastAR, Meta, Laster SeeThru та K-Glass.



2 СТВОРЕННЯ ПРОСТИХ AR-ПРОГРАМ ЗА ДОПОМОГОЮ A-FRAME ТА AR.JS

2.1. Налаштування проекту та створення сцени

A-Frame дуже схожий на HTML – усі команди описуються тегами, які подібні до тегів HTML, але, на відміну від останніх, інтерпретуються не у веб-браузері на боці клієнта, а є способом доступу до JavaScript, що виконується на боці сервера. Разом із AR.js він є потужним API для AR, що приховує деталі реалізації мовою JavaScript.

Для початку роботи з A-Frame необхідно перейти на сайт A-Frame за посиланням <https://aframe.io/docs/1.0.0/introduction/installation.html> та завантажити з нього файл збірки JavaScript (JS Build, Production Version – <https://aframe.io/releases/1.0.4/aframe.min.js>).

Завантажений файл збірки необхідно зберегти у окремому каталозі (наприклад, aframe) всередині робочого каталогу, після чого створити у останньому індексний файл HTML ARindex.html:

```
code
├── ARindex.html
└── aframe
    └── aframe.min.js
```

із наступним вмістом

```
<!DOCTYPE html>
```

```

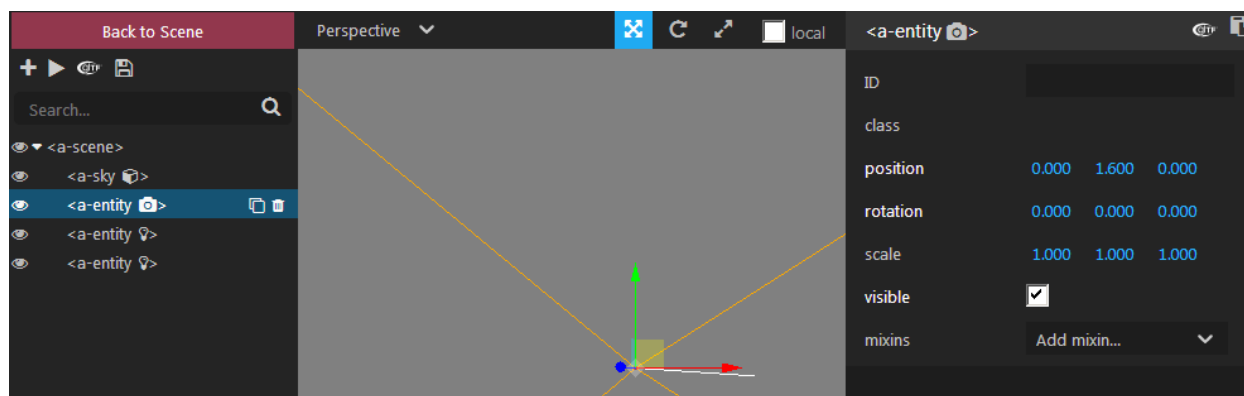
<html>
  <script src="aframe/aframe.min.js"></script>
  <a-scene>
    <a-sky color="grey"></a-sky>
  </a-scene>
</html>

```

Файл містить команду `script` із посиланням на завантажений файл – вона завжди використовується для початку роботи з бібліотекою JavaScript. Саме у ньому інтерпретується тег `a-scene`, в якому знаходитиметься більша частина коду A-Frame. Тег `a-sky` створюватиме фоновий колір (його також можна застосувати для розміщення 360° зображення на сцені). Після відкриття файлу `ARindex.html` у веб-браузері отримаємо вікно із сірим фоном.

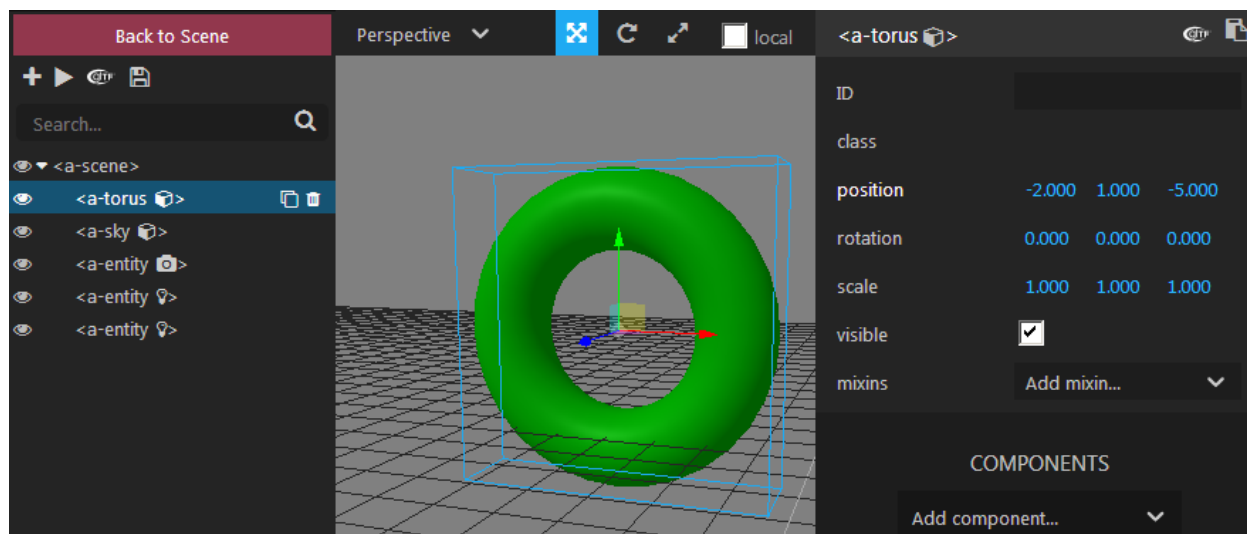


Про те, що A-Frame API дійсно працює, свідчить режим VR (Virtual Reality) у нижньому правому куті. Суттєво більше відомостей можна отримати, увімкнувши режим візуального 3D-інспектора (Ctrl-Alt-I).



Додавши до індексного файлу команду

`<a-torus position="-2 1 -5" color="green" radius="1.2"></a-torus>`
отримаємо зображення зеленого тору на сірому тлі.



У новостворених об'єктів є певні атрибути – ознайомитись із ними можна у документації до A-Frame. Поточна версія підтримує наступні примітиви:

- a-box – прямокутний паралелепіпед;
- a-camera – камера (визначає, що бачить користувач);
- a-circle – круг;
- a-collada-model – 3D-модель у форматі COLLADA (.dae);
- a-cone – конус;
- a-cursor – опрацювання подій від миши;
- a-curvedimage – панорамне зображення;
- a-cylinder – циліндричні поверхні;
- a-dodecahedron – дванадцятигранник;
- a-gltf-model – 3D-модель у форматі glTF (.gltf);
- a-icosahedron – двадцятигранник;
- a-image – пласке зображення;
- a-light – джерела світла;
- a-link – гіперпосилання;
- a-obj-model – 3D-модель у форматі Wavefront (.obj/.mtl);

`a-octahedron` – восьмигранник;
`a-plane` – площина;
`a-ring` – пласке кільце або диск;
`a-sky` – додає фоновий колір або 360° зображення;
`a-sound` – джерело звуку;
`a-sphere` – сфера або багатогранник;
`a-tetrahedron` – трикутна піраміда;
`a-text` – плаский текст;
`a-torus-knot` – кренделеподібна фігура;
`a-torus` – тор;
`a-triangle` – трикутна поверхня;
`a-video` – відео як текстура на площині;
`a-videosphere` – 360° фонове відео.

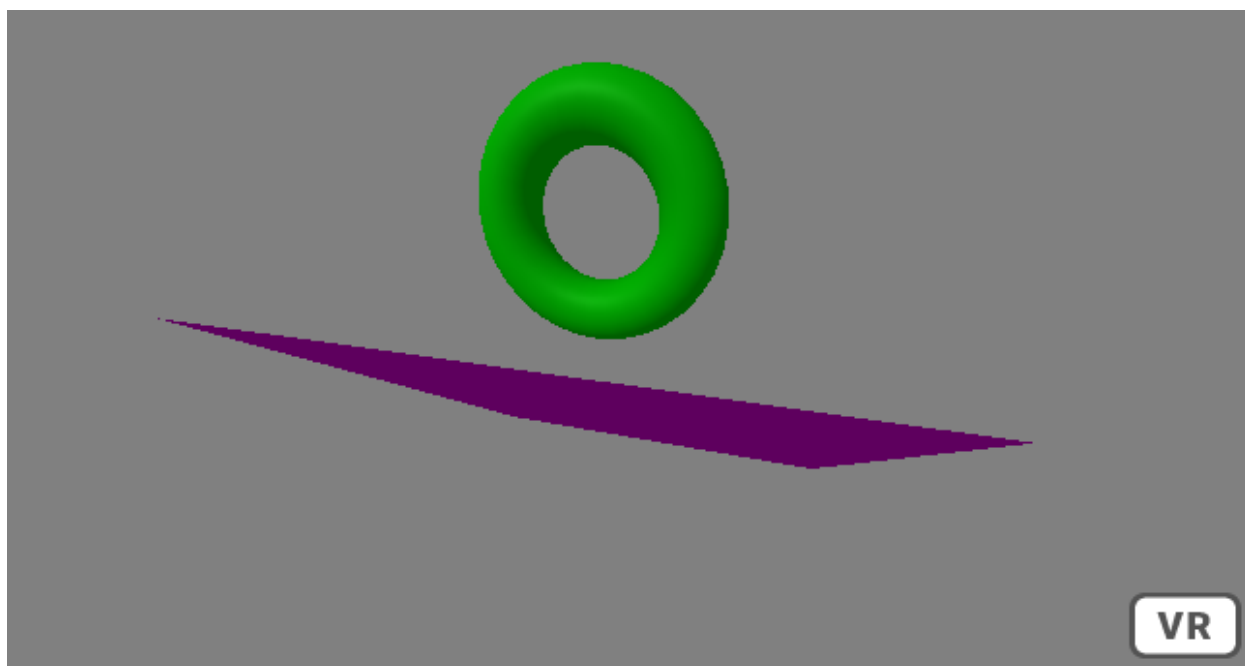
2.2. Об'єктні сітки та атрибути сітки

Додамо до попередньої сцени, що містить зелений тор та сірий фон, ще кілька примітивів. Спочатку вставимо між тором та фоном площину:

```
<a-plane width="7" height="7" rotation="50 0 0"
      position="3 -2 -3" color="purple"></a-plane>
```

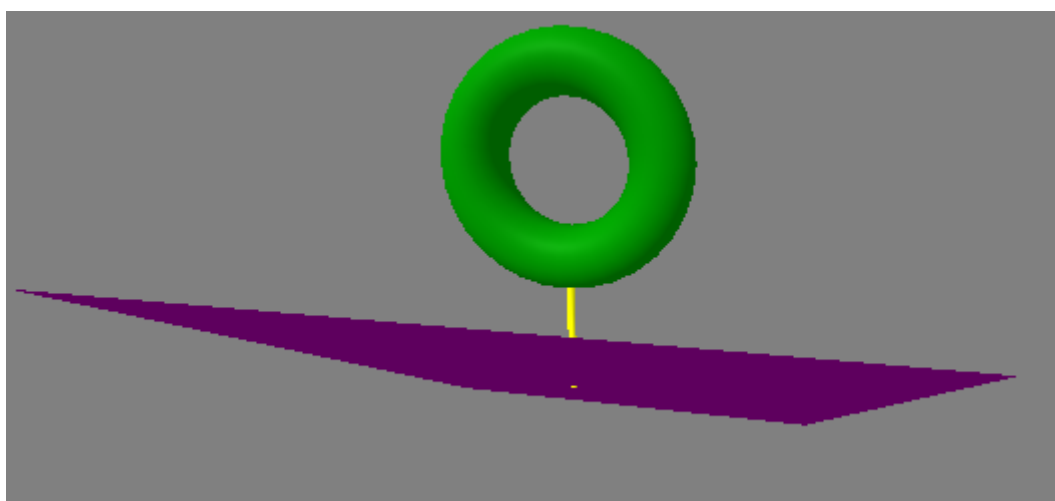
Для перегляду параметрів `a-plane` слід скористатись документацією за посиланням <https://aframe.io/docs/1.0.0/primitives/a-plane.html> – тут можна знайти різні атрибути площини, які можна застосувати до неї в тегу A-Frame. Параметри `width` та `height` відповідають за ширину та висоту прямокутника, `rotation` вказує на необхідність її повороту на 50° відносно вісі `x` та на 0 – відносно `y` та `z`, `position` – координати початку площини за відповідними осями, а `color` – обраний колір.

Площину, налаштовану у такий спосіб, на сцені можна побачити лише в режимі інспектора, тому що вона розміщена дуже далеко на задньому плані. Якщо змінити координати початку площини на `"-2 -2 -5"`, її можна побачити.



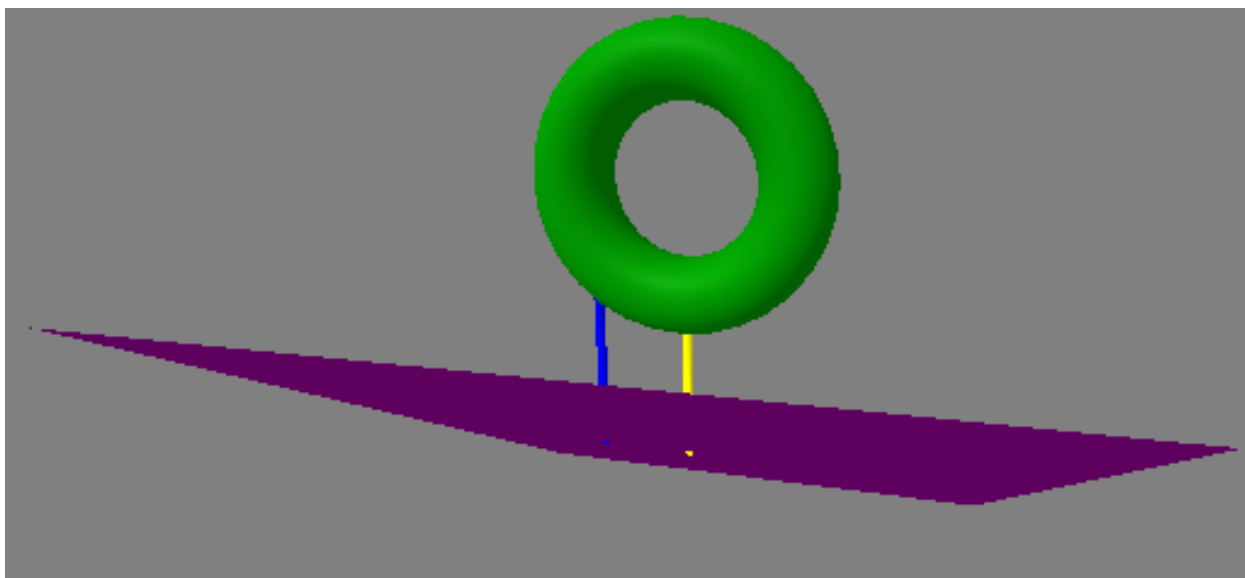
Отже, якщо в певній позиції об'єкт фактично не відображається, це може бути проблемою з позиціонуванням самого об'єкту всередині коду. Тепер, коли у нас є площина, додамо ще кілька об'єктів. Створимо циліндричний об'єкт у формі льодяника жовтого кольору, розташованого під самим тором

```
<a-cylinder color="yellow" height="2" radius="0.05"
      position="-2 -1 -5"></a-cylinder>
```



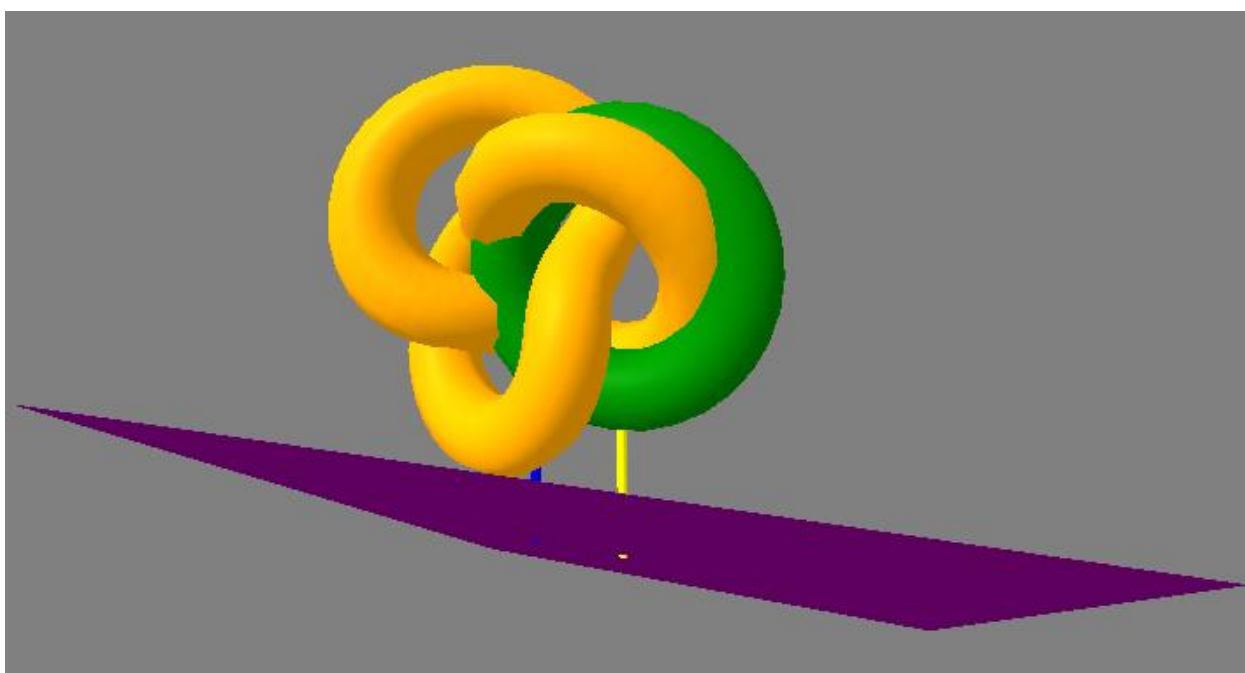
Ще один циліндр:

```
<a-cylinder color="blue" height="2" radius="0.05"
      position="-3 -1 -5"></a-cylinder>
```



I, нарешті, кренделеподібна фігура:

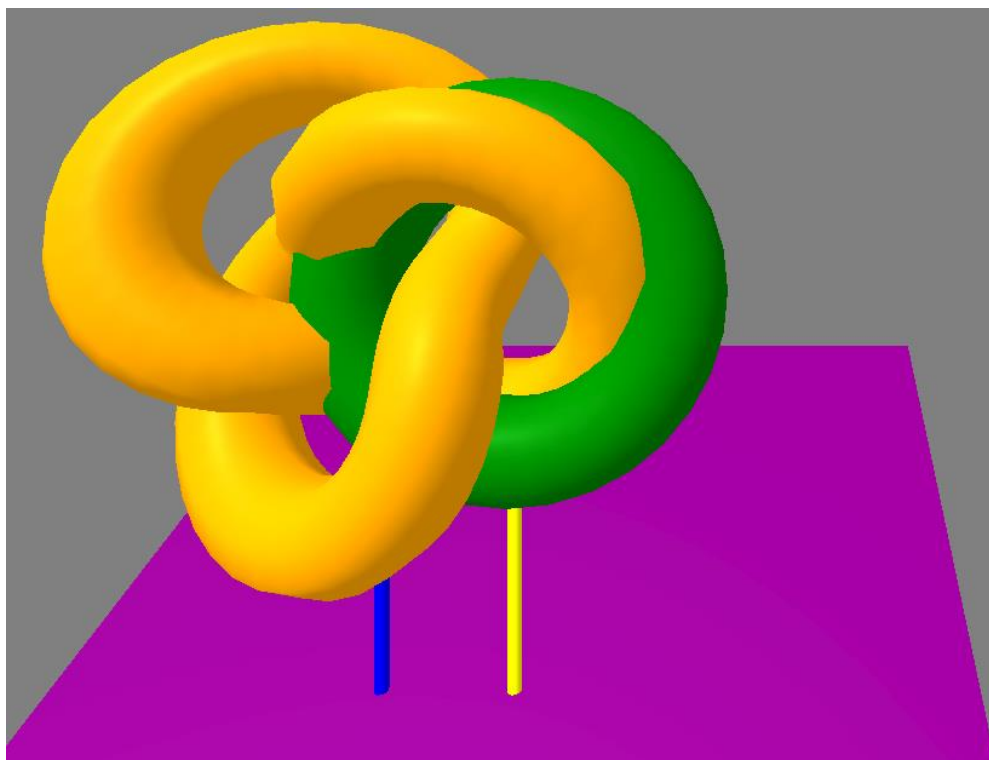
```
<a-torus-knot color="orange" radius="1.2"
      position="-3 1 -5"></a-torus-knot>
```



Досить цікава форма – без документації не розібратись.

2.3. Додавання тексту та анімації до об'єктів

Виконаємо невеликі зміни у індексному файлі в атрибуті обертання площини – зміна кута огляду на "-55 0 0" надає їй суттєво кращого вигляду.

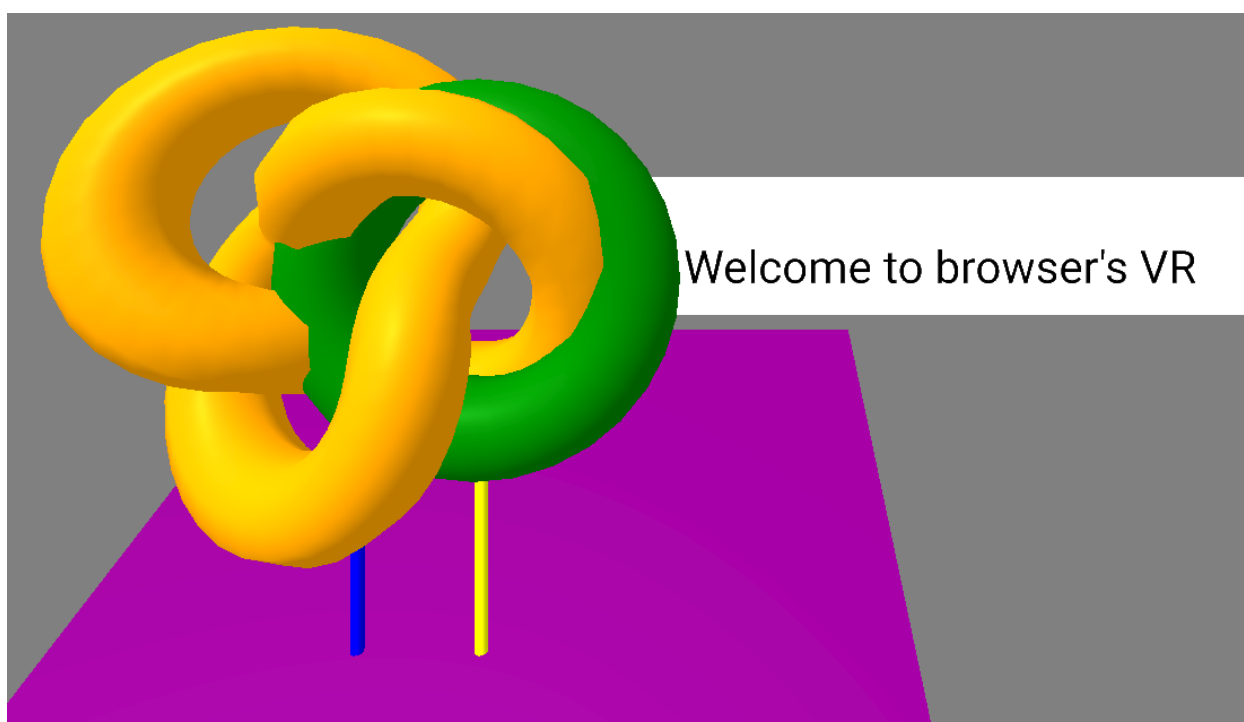


Додамо до сцени площину, перпендикулярну попередній:

```
<a-plane width="9" height="2" position="3 1 -9"></a-plane>
```

За замовчанням її колір буде білим – достатньо зручний для того, щоб перед ним розмістити напис:

```
<a-text value="Welcome to browser's VR" color="black"
      width="10" position="-0.5 1 -6"></a-text>
```



«Лобова» спроба замінити даний напис на кириличний, скоріше за все,

виявиться невдалою без використання відповідного шрифту. Більш ніж 2000 різних шрифтів та способи їх використання можна знайти у репозитарії <https://github.com/etiennepinchon/aframe-fonts>. Будь-який шрифт із репозитарію може бути підключений за посиланням виду `https://raw.githubusercontent.com/etiennepinchon/aframe-fonts/master/fonts/[FONT_NAME]/[FONT_TYPE].json`

Інший варіант – створення власного шрифту за допомогою MSDF font generator:

MSDF font generator

[SOURCE](#) [ISSUES](#)

1. Select font

Default font is Microsoft YaHei, which supports several languages. Optionally, upload another (.ttf) font:

Upload a font: arial.ttf

2. Select character set

qwertyuiopasdfghjklzxcvbnmQWERTYUIOPASDFGHJKLZXCVCBNМйцукенгшщзхїфівапролджєячсмитьбю`1234567890-~\!;'.<>?*"~!@#\$%^&'()

3. Create MSDF font

arial

256px (default)

The generated file will be named `arial-msdf.json`.

4. Preview and download files

Preview shows only first five characters of charset.

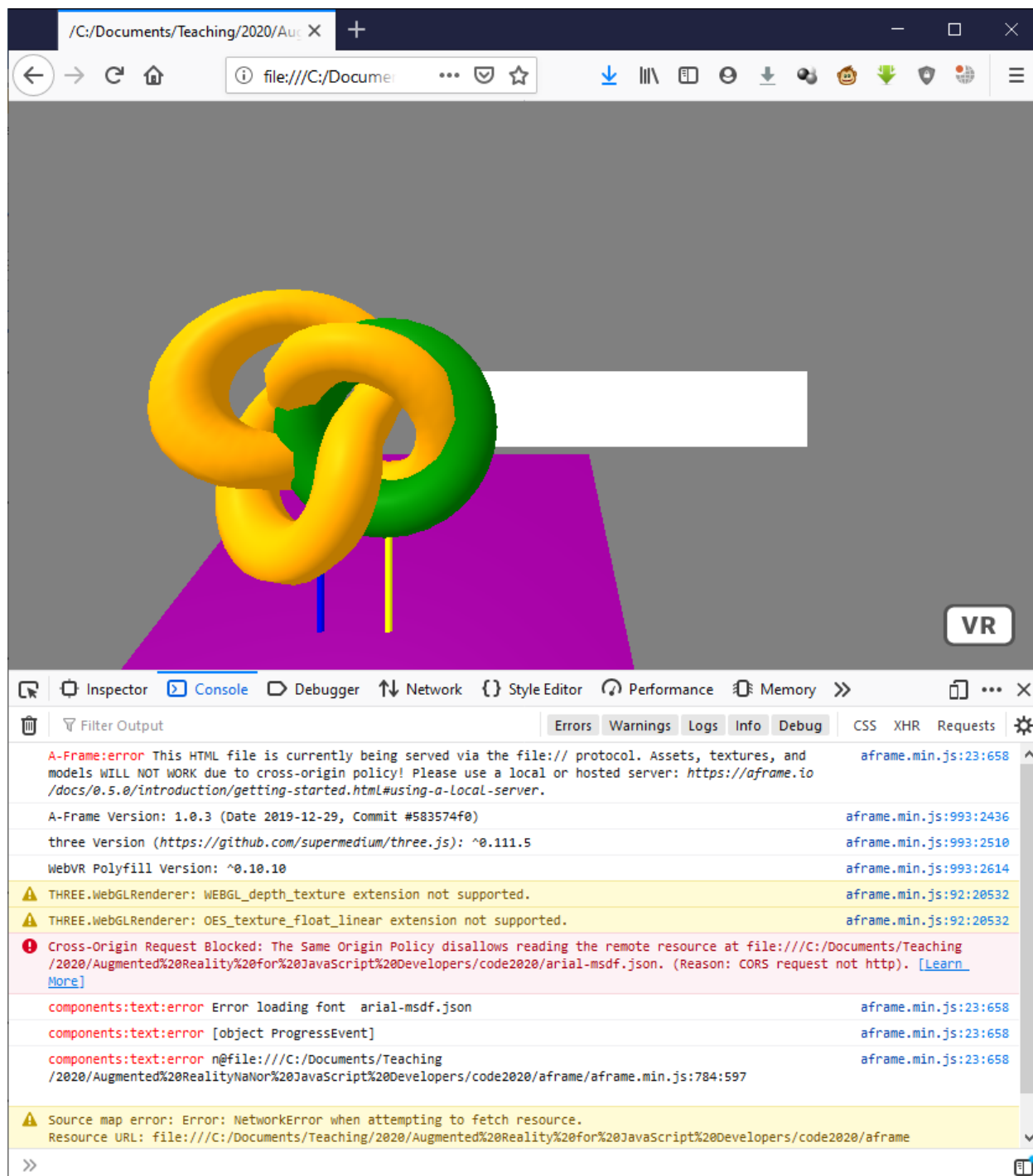
Архів, завантажуваний з <https://msdf-bmfont.donmccurdy.com>, містить шрифт у форматах JSON та PNG (для вихідного шрифту arial.ttf це будуть arial-msdf.json та arial.png відповідно – розташуйте їх там само, де знаходиться й ARindex.html). Чим більше символів обирається, тим більше має бути розмір PNG-файлу (до 1024x1024). Для економії трафіку до них включені лише ті

символи, які були обрані користувачем – всі інші не відобразяться.

Внесемо зміни до параметру `value` та додамо параметри `font` і `negate`:

```
<a-text value="Вітаємо у браузерній VR!" color="black"
width="10" position="-0.5 1 -6"
font="arial-msdf.json" negate="false"></a-text>
```

Якщо останній матиме істинне значення, обраний колір стане фоновим для кожної з літер.



Новий текст не відображається – згідно повідомлень із консолі браузера,

локальне завантаження (за протоколом `file://`) шрифту (так само, як і багатьох інших ресурсів), не дозволено. Для того, щоб виправити цю помилку, необхідно файли шрифту розмістити на сервері з доступом за протоколом HTTP або HTTPS. Де та як це зробити, суттєво залежить від власних можливостей користувача – студенти зазвичай можуть звернутись до викладача для того, щоб отримати доступ до зовнішнього серверу, або налаштувати власний.

Наприклад, для отримання доступу за протоколом FTP до власного розділу на сервері `https://playground2.ccjournals.eu` користувачеві необхідні знати ім'я або адресу FTP-серверу (host, hostname), ім'я (login, username) та пароль (password).

net2ftp a web based FTP client

HOME SCREENSHOTS FEATURES » DOWNLOAD HELP » ABUSE PRIVACY »

Home

Login *Connect to your FTP server and start editing your website now.*

Basic FTP login

FTP server
ftp.ccjournals.eu 21

Username
student01@playground2.ccjournals.eu

Password
●●●●●●●●●●

[Privacy notices](#)
Please enter your email address as identifier to give you the right of access and erasure:
semerikov@gmail.com Enter your email address

☒ I agree to the [Disclaimer](#), [Privacy Policy](#) and [Cookie Policy](#)

Login

Після реєстрації користувач побачить поточний вміст його розділу на сервері:

ftp.ccjournals.eu    

/

Directory Tree: root /

Files which are too big can't be downloaded, uploaded, copied, moved, searched, zipped, unzipped, viewed or edited; they can only be renamed, chmodded or deleted.

[New dir](#) [New file](#) [Upload](#) [Java Upload](#) [Install](#) [Advanced](#)

Transform selected entries: [Copy](#) [Move](#) [Delete](#) [Rename](#) [Chmod](#) [Download](#) [Zip](#) [Unzip](#) [Size](#) [Search](#)

All	Name	Type	Size	Owner	Group	Perms	Mod Time			
	 Up..									
☐	 AR.js-master	Directory	4096	ccjourna	ccjourna	rw-r--r--	Feb 15 13:31			
☐	 aframe	Directory	27	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30			
☐	 three.js-dev	Directory	316	ccjourna	ccjourna	rw-r--r--	Feb 15 13:34			
☐	 .ftpquota	FTPQUOTA File	15	ccjourna	ccjourna	rw-----	Feb 15 14:18	View	Edit	Open
☐	 AR.js-master.zip	Zip archive	136626749	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30			Open
☐	 ARindex.html	HTML file	179	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30	View	Edit	Open
☐	 index.html	HTML file	831	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30	View	Edit	Open
☐	 photo.jpg	JPEG file	39204	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30	View	Edit	Open
☐	 test.html	HTML file	403	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30	View	Edit	Open
☐	 three.js-dev.zip	Zip archive	266302404	ccjourna	ccjourna	rw-r--r--	Feb 15 13:30			Open

У файлі index.html доцільно розмістити посилання на інші файли, створені користувачем. Редагувати їх можна як безпосередньо на сервері, так й локально із подальшим завантаженням. Для того, щоб поточний приклад був працездатним, необхідно завантажити як сам ARindex.html, так й конвертовані файли із шрифтами arial-msdf.json та arial.png:



Upload files and archives



Upload to directory: 

Files

Files entered here will be transferred to the FTP server.

[ARindex.html](#)
 [arial-msdf.json](#)
 [arial.png](#)
 No file selected.

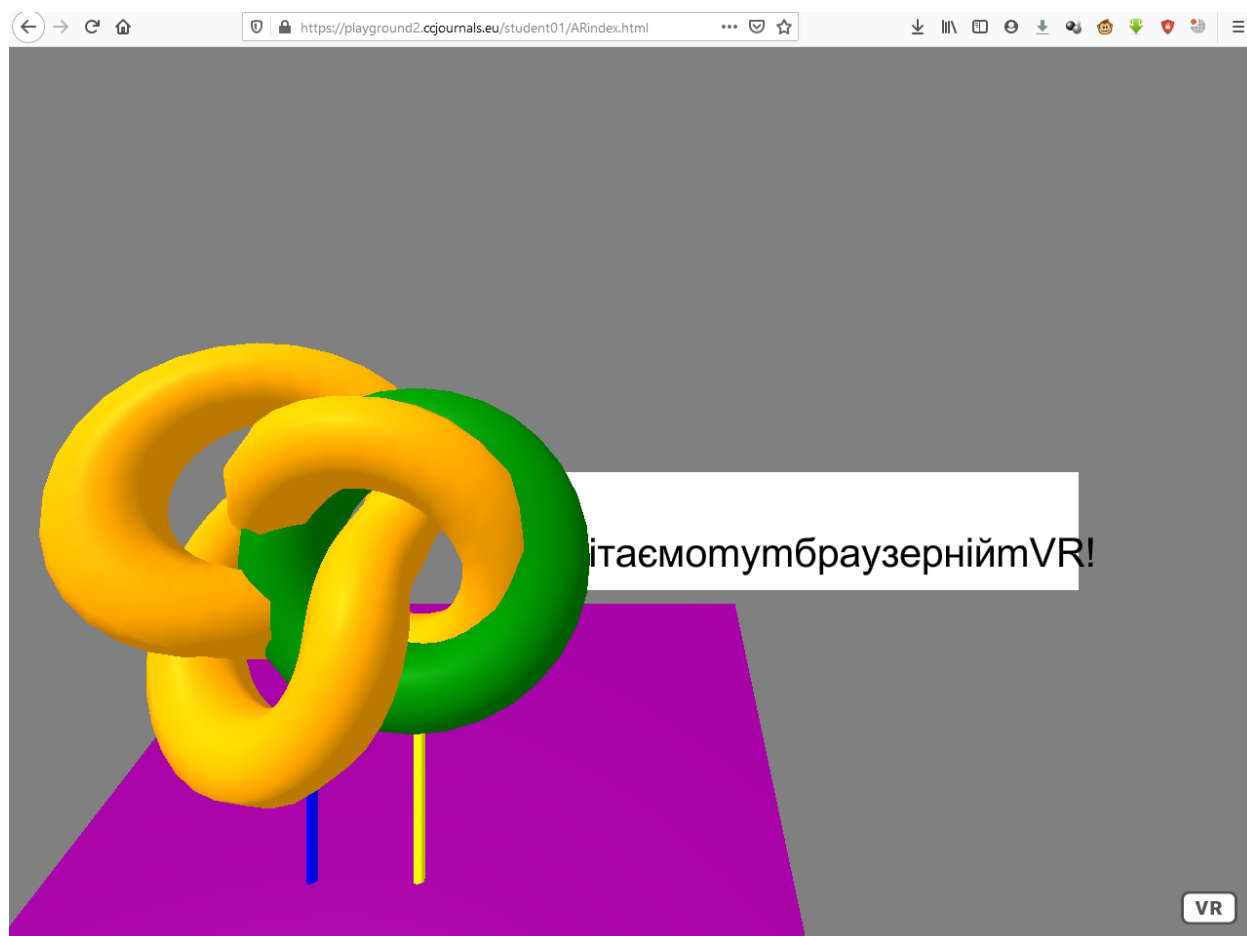
Archives (zip, tar, gzip, gz)

Archives entered here will be decompressed, and the files inside will be transferred to the FTP server.

No file selected.

Для масового завантаження файлів доцільно скористатись можливістю завантаження архіву з ними.

Перевірка виконання ARindex.html на сервері показує, що результат вже суттєво кращий:



Для завершення локалізації надпису необхідно масштабувати його до розміру площини, вказавши параметр `scale`, та додати до набору символів, для яких генерується шрифт, пропуск. Застосування символів, що не входять до набору ASCII, потребує додаткового вказання одного з найпоширеніших кодувань тексту – UTF-8 – після тегу `<html>`:

```
<meta charset="utf-8">
```

Для того, щоб анімувати примітив, необхідно додати до нього параметр `animation`.

```
<a-torus position="-2 1 -5" color="green" radius="1.2"
  animation="property: components.material.material.color;
  type: color; from: green; to: red; loop: true; dur: 10000">
</a-torus>
```

У даному прикладі основним в параметрі `animation` є `property` – та складова об'єкту, яка буде змінюватись

(`components.material.material.color` – колір). `dur` визначає тривалість анімації в мілісекундах (10000 мс = 10 с), `from` задає початкове значення атрибуту, `to` – кінцеве, а `loop` визначає кількість повторень циклу зміни (`true` відповідає нескінченному циклу).

Наступний приклад демонструє зміну атрибуту `rotation` для того, щоб змусити тороподібний об'єкт обертатися навколо центру:

```
<a-torus-knot color="orange" radius="1.2" position="-3 1 -5"
  animation="property: rotation; to: 0 0 360; loop: true; dur: 10000">
</a-torus-knot>
```

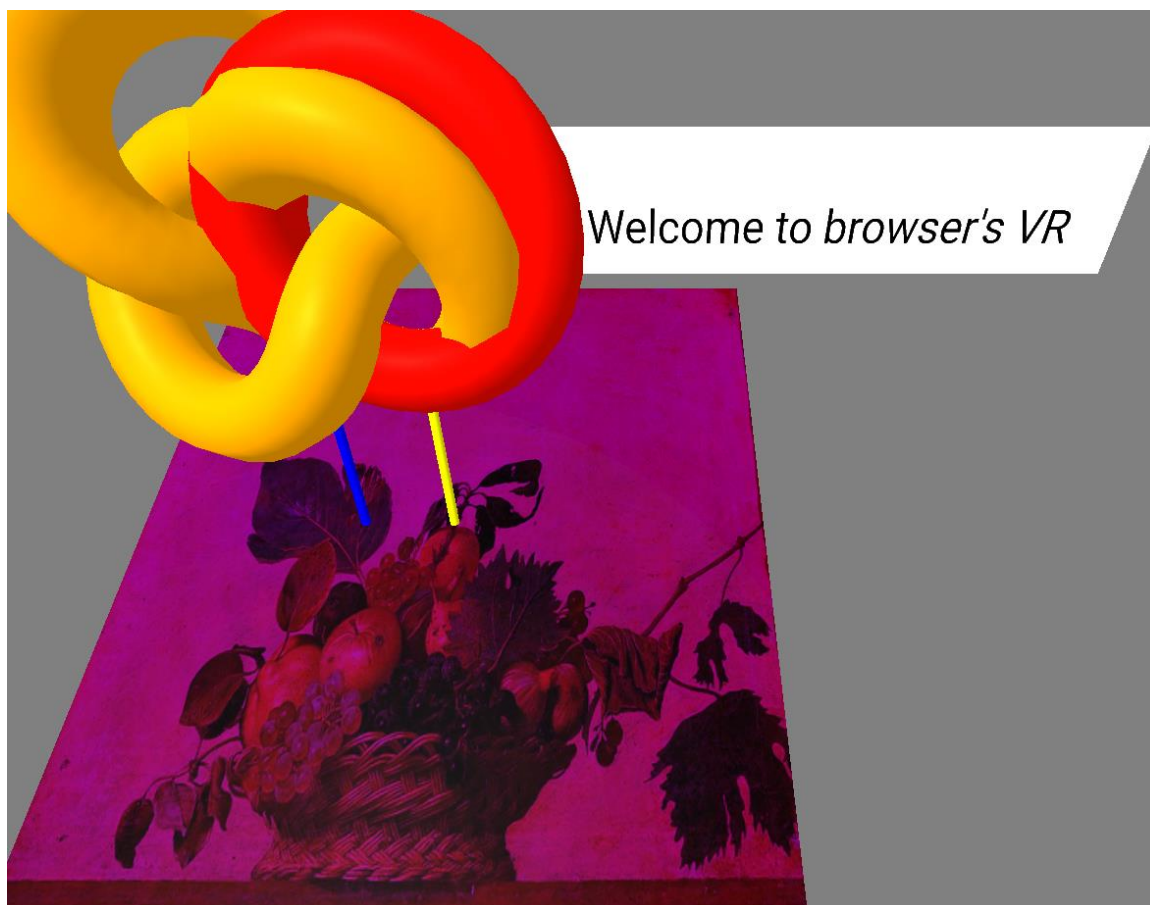
Більш детально про параметр `animation` можна прочитати у довідці з A-Frame Core API за посиланням <https://aframe.io/docs/1.0.0/components/animation.html>

2.4. Додавання текстур до об'єктів

Головна сторінка <http://aframe.io> містить посилання на вихідні коди бібліотеки. На рівень вище серед 18 репозиторіїв A-Frame можна знайти `sample-assets` – приклади файлів зображень, моделей та аудіозаписів для використання у A-Frame. Для того, щоб скористатися ними, необхідно отримати пряме посилання на об'єкт репозитарію, та додати його у якості атрибуту `src` до об'єкту, для якого обране зображення виступатиме текстурою. Змінимо площину під торами:

```
<a-plane
src="https://raw.githubusercontent.com/aframevr/sample-
assets/master/assets/images/illustration/758px-
Canestra_di_frutta_(Caravaggio).jpg" width="7" height="7"
rotation="-55 0 0" position="-2 -2 -5" color="purple"></a-plane>
```

Посилання на файл текстури може бути узятé із будь-якого місця, але не завжди сайти дозволяють використовувати прямі посилання на файли сайту поза його межами. «Політика того ж походження» (`same origin policy`) є важливим механізмом безпеки у сучасних веб-браузерах, що стосується як виконуваних у браузері файлах, так й використовуваних.



2.5. Додавання AR.js до програми

Перш за все необхідно перейти до сховища AR.js у GitHub за посиланням <https://github.com/jeromeetienne/AR.js> і завантажити ZIP-файл, натиснувши «Clone or download». Станом на початок 2020 року розмір репозитарію AR.js – більше 170 Мб, тому завантаження архіву може тривати певний час.

jeromeetienne / AR.js

Watch 579 Star 14.7k Fork 2k

Code Issues 85 Pull requests 2 Actions Projects 1 Wiki Security Insights

Efficient Augmented Reality for the Web - 60fps on mobile!

a-frame three.js webar

1,589 commits 16 branches 0 packages 37 releases 45 contributors MIT

Branch: master New pull request Find file Clone or download

nicolcarpignoli Last commit before release Latest commit 32532d7 12 days ago

.github/ISSUE_TEMPLATE	Update issue templates	13 months ago
aframe	Last commit before release	12 days ago
data	clean a lot of old stuff	2 months ago
test	changed THREE.AxisHelper to THREE.AxesHelper in examples	10 months ago
three.js	Last commit before release	12 days ago

Завантажений архів доцільно розпакувати у каталог, де зберігаються індексний файл та каталог aframe:

```

  ARindex.html
  └── aframe
      aframe.min.js
  └── AR.js-master
      .gitignore
      .travis.yml
      AR.js-1920-1080-HD.png
      CHANGELOG.md
      CODE_OF_CONDUCT.md
      CONTRIBUTING.md
      HOW_TO_RELEASE.md
      ISSUE_TEMPLATE.md
      LICENSE.txt
      Makefile
      package-lock.json
      package.json
      PULL_REQUEST_TEMPLATE.md
      README.md
      TODO.md
      └── .github
          └── ISSUE_TEMPLATE
              bug_report.md
  ...

```

Структура каталогів репозитарію досить розгалужена, але прозора. Так, у `AR.js-master\data\images\` можна знайти файл `HIRO.jpg` – дане зображення буде маркером, необхідним для того, щоб при наведенні на нього веб-камери відображалась сцена.



Для початку використання `AR.js` необхідно внести кілька змін до вихідного індексного файлу `ARindex.html`. По-перше, після тегу `script` для підключення стандартної версії бібліотеки `A-Frame` додається ще один тег для роботи із `AR.js` та захоплення даних із веб-камери:

```
<script src="AR.js-master/aframe/build/aframe-ar.min.js"></script>
```

Для того, щоб побудовані об'єкти краще відображались на маркері, змінимо деякі їх атрибути:

- позицію тора встановимо у (0; 0,5; 0), а радіус – у 0,5;
- ширину та висоту площини зменшимо до 3,5, а позицію встановимо у (0; -1; 0);
- позицію першого циліндра змінімо на (0; 0; 0), а другого – на (1; 0; 0);
- для кренделеподібної фігури радіус встановимо у 0,5, а позицію – у (1; 0,5; 0);
- до площини, на яку накладатиметься текст, застосуємо текстуру за посиланням https://raw.githubusercontent.com/aframevr/sample-assets/master/assets/images/uvgrid/UV_Grid_Sm.jpg та встановимо її ширину у 2,5, висоту – в 1,5, а позицію – у (0; 1; -1);
- ширину тексту встановимо у 3, а позицію – у (-1; 0,5; -1).

Створена сцена була побудована для використання у веб-VR. Для використання об'єктів на ній у доповненій реальності спочатку позбавимось сірого фону, доданого тегом `a-sky`, шляхом його видалення. Далі необхідно вбудувати у сцену `AR.js` шляхом додавання атрибутів `embedded` (застосовувати вбудований розпізнавач маркерів) та `arjs = 'trackingMethod: best;'` (використовувати найкращий метод відстеження):

```
<a-scene embedded arjs = 'trackingMethod: best;'>
```

Крім того, доведеться створити тег `a-anchor` для того, щоб виконати прив'язку до доповненої реальності:

```
<a-anchor hit-testing-enabled='true'>
```

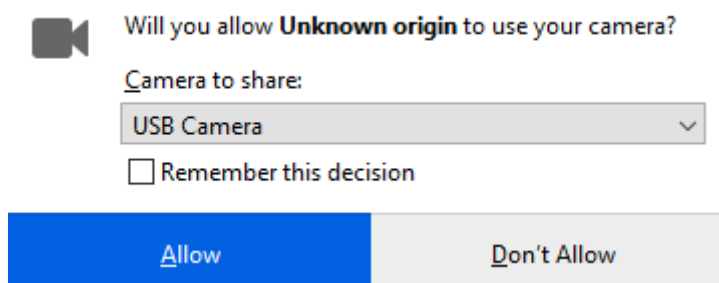
Всі об'єкти, розміщені на сцені, слід розмістити між тегами `<a-anchor>` та `</a-anchor>`. Останній крок – додавання статичної камери:

```
<a-camera-static/>
```

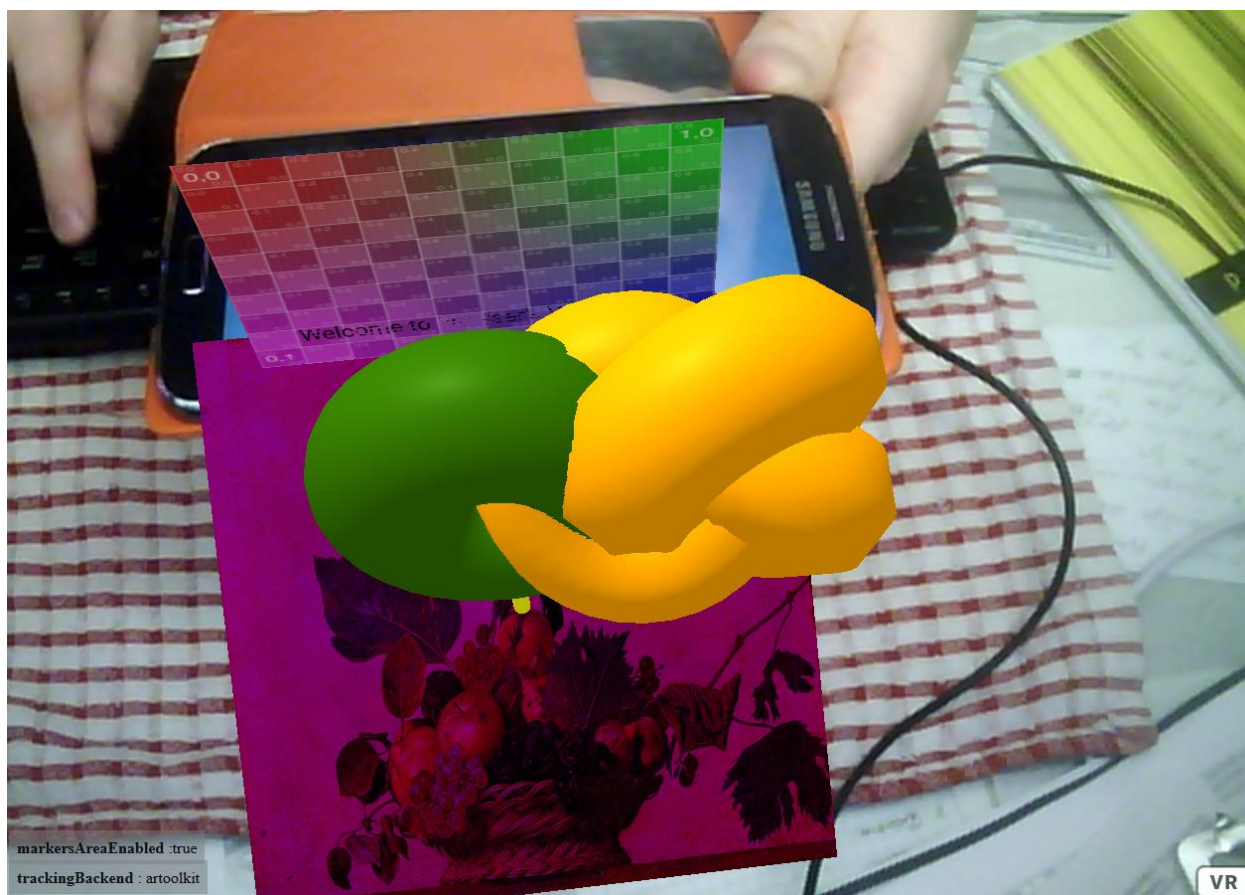
На відміну від попередніх, це тег не є парним, що, за стандартом XHTML, потребує додавання слешу наприкінці тега.

```
<!DOCTYPE html>
<html>
  <meta charset="utf-8">
  <script src="aframe/aframe.min.js"></script>
  <script src="AR.js-master/aframe/build/aframe-ar.min.js"></script>
  <a-scene embedded arjs = 'trackingMethod: best;'>
    <a-anchor hit-testing-enabled='true'>
      <a-torus position="0 0.5 0" color="green" radius="0.5"
        animation="property: components.material.material.color; type: color; from: green; to: red; loop: true; dur: 10000"></a-torus>
      <a-plane width="3.5" height="3.5" rotation="-55 0 0" position="0 -1 0" color="purple" src=
        "https://raw.githubusercontent.com/aframevr/sample-assets/master/assets/images/illustration/758px-Canestra_di_frutta_(Caravaggio).jpg"
      ></a-plane>
      <a-cylinder color="yellow" height="2" radius="0.05" position="0 0 0"></a-cylinder>
      <a-cylinder color="blue" height="2" radius="0.05" position="1 0 0"></a-cylinder>
      <a-torus-knot color="orange" radius="0.5" position="1 0.5 0" animation="property: rotation; to: 0 0 360; loop: true; dur: 10000">
      </a-torus-knot>
      <a-plane width="2.5" height="1.5" position="0 1 -1"
        src="https://raw.githubusercontent.com/aframevr/sample-assets/master/assets/images/uvgrid/UV_Grid_Sm.jpg"></a-plane>
    <!--
      <a-text value="Вітаємо у браузерній VR!" color="black" width="10" position="-0.5 1 -6"
        font="arial-msdf.json" negate="false"></a-text> -->
      <a-text value="Welcome to browser's VR" color="black" width="3" position="-1 0.5 -1"></a-text>
    </a-anchor>
    <a-camera-static/>
  </a-scene>
</html>
```

Після відкриття індексного файлу у веб-браузері, за наявності веб-камери, необхідно надати дозвіл на доступ до неї.



Об'єкти на сцені з'являться, коли камера буде спрямована на маркер.



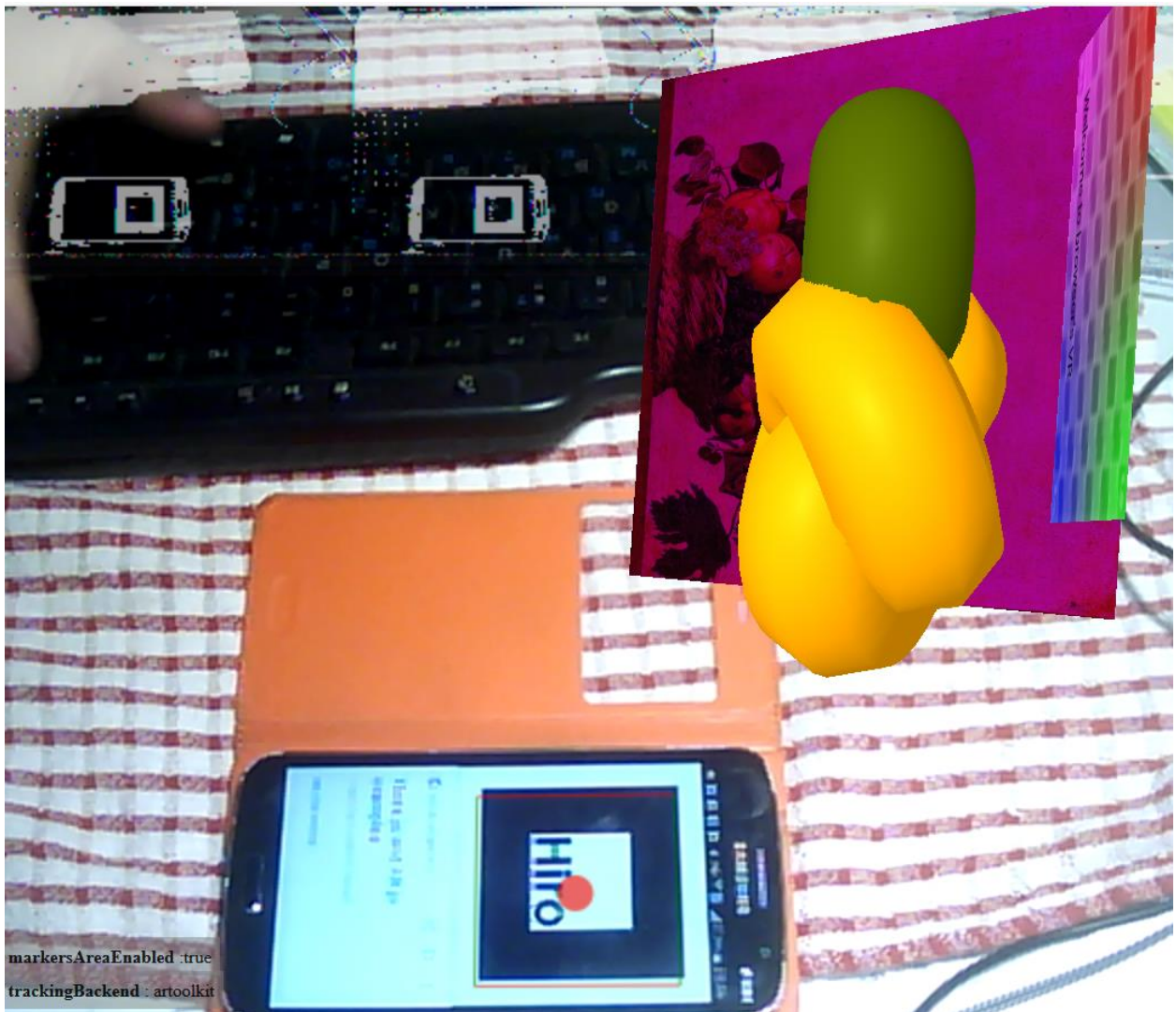
Файл маркеру за замовчанням можна відкрити на будь-якому мобільному пристрої або роздрукувати.

До параметру `arjs` можуть бути передані наступні значення:

`trackingMethod` – спосіб відстежування маркерів (за замовчанням `'best'`);

`debugUIEnabled` – показувати додаткові відомості для налагодження (за замовчанням `true`);

`debug` – вмикає налагоджувальний режим (за замовчанням `false`);



`detectionMode` – тип маркеру (можливі `'color'`, `'color_and_matrix'`, `'mono'`, `'mono_and_matrix'`);

`matrixCodeType` – тип матричного коду для `detectionMode` `'color_and_matrix'` та `'mono_and_matrix'` (можливі `3x3`, `3x3_HAMMING63`, `3x3_PARITY65`, `4x4`, `4x4_VCH_13_9_3`, `4x4_VCH_13_5_5`);

`patternRatio` – співвідношення сторін для користувацьких маркерів (за замовчанням не використовується: -1);

`cameraParametersUrl` – посилання на параметри камери (наприклад, "AR.js-master\data\data\camera_para.dat");

`maxDetectionRate` – максимальна частота, з якою бібліотека намагається знайти маркер на зображенні з камери (за замовчанням не використовується: -1);

`sourceType` – джерело зображення (можливі 'webcam', 'image', 'video');

`sourceUrl` – посилання на джерело (якщо `sourceType` – 'image' або 'video');

`sourceWidth` та `sourceHeight` задають роздільну здатність вихідного зображення (наприклад, 640x480), `displayWidth` та `displayHeight` – відповідно відображуваного зображення;

`deviceId` – опціональний ідентифікатор камери;

Більшість із цих значень використовує найстаріша бібліотека для доповненої реальності – ARToolKit, явне використання якої може бути задано параметром `artoolkit` примітиву `a-scene`. Детальні керівництва та приклади створення веб-AR за допомогою AR.js можна знайти у репозитарії його автора – Жерома Етьєнна.

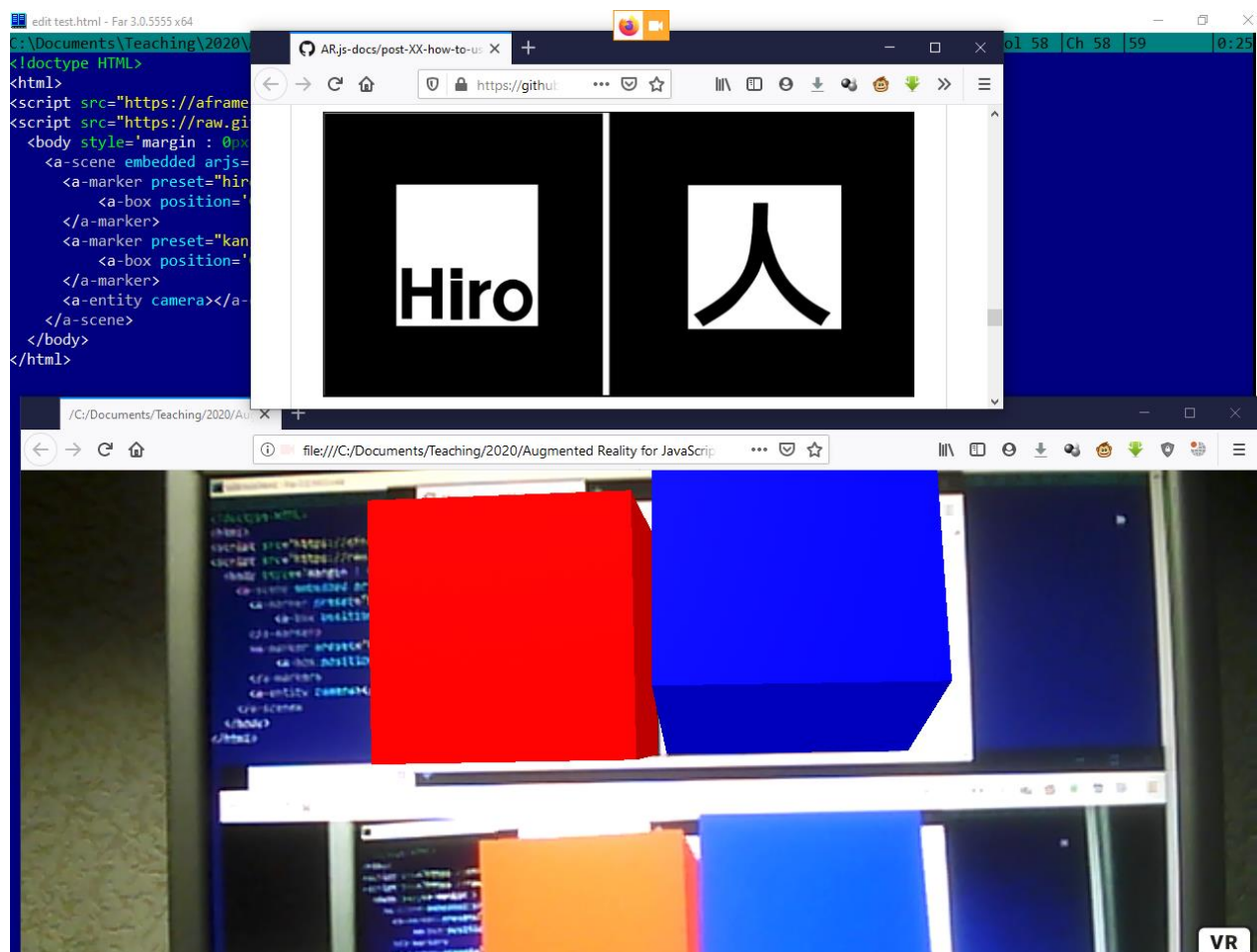
2.6. Базовий JavaScript та об'єктна модель A-Frame

Використовуючи тег `script` із атрибутом `src`, бібліотеки (і взагалі будь-який текст мовою JavaScript) можна включати до індексного файлу як з локального розташування, так й з віддаленого. Наприклад, замість завантаження бібліотек A-Frame та AR.js можна було б послатися на їх розташування в мережі Інтернет. Якщо тег `a-anchor` виконує загальну прив'язку групи об'єктів, що він їх охоплює, до маркеру, то у даному прикладі явно вказано, який маркер буде використовуватись, за допомогою тегу `a-marker`: у такий спосіб різним маркерам можна співставити різні групи

об'єктів:

```
<!doctype HTML>
<html>
<script src="https://aframe.io/releases/1.0.4/aframe.min.js"></script>
<script src="https://raw.githubusercontent.com/jeromeetienne/AR.js/2.1.8/aframe/build/aframe-ar.js"></script>
  <body style='margin : 0px; overflow: hidden;'>
    <a-scene embedded arjs="debugUIEnabled:false">
      <a-marker preset="hiro">
        <a-box position='0 0.5 0' material='color: red;'></a-box>
      </a-marker>
      <a-marker preset="kanji">
        <a-box position='0 0.5 0' material='color: blue;'></a-box>
      </a-marker>
      <a-entity camera></a-entity>
    </a-scene>
  </body>
</html>
```

У даному прикладі використовуються два стандартні маркери – "hiro" та "kanji".



Крім того, одинарний тег `<a-camera-static/>` замінено на об'єкт `camera` за допомогою пари тегів `<a-entity camera></a-entity>`. Зауважимо, що, хоча у цьому прикладі різниця несуттєва, дана заміна не є однозначною. Головна її мета – показати, що JavaScript є об'єктно-

орієнтованою мовою. Будь-який приклад, з якого розпочинається використання цієї мови, яскраво це ілюструє:

```
<!DOCTYPE html>
<html> <body>
    <script type="text/javascript">
        document.write("<font size=7>Hello World!</font>");
    </script>
</body>
</html>
```

Тег `script` доповнений атрибутом `type`, у якому конкретизовано мову. Програма складається з одного єдиного рядка, в якому з об'єкту `document` викликається метод `write`, параметром якого є рядок, що з'являється у вікні браузера.

Hello World!

Для початку застосування JavaScript достатньо знати мінімальний синтаксис цієї мови:

1. *Змінні* у JavaScript створюються з літер, цифр, знаків долару та підкреслення:

а) при першому зверненні до них:

```
x = 1 // створити змінну x та надати їй значення 1
```

б) за допомогою ключового слова `var`

так:

```
var x // створити змінну x
x = 1 // надати значення змінній
```

або так

```
var x = 1 // створити змінну x та надати їй значення 1
```

Змінні, створені без використання `var`, стають глобальними. Змінні також можуть бути оголошені за допомогою `let` (для змінної рівня блоку) та `const` (для сталої).

2. *Коментарі* створюються аналогічно до C++:

```
// однорядковий коментар
/*
    багаторядковий
    коментар
*/
```

3. Прості типи даних:

– рядковий (`string`) – визначається подвійними або одинарними лапками і використовується для символьних даних;

– числовий (`number`) – визначається відсутністю лапок і використовується для дійсних чисел (наприклад, 345 – ціле десяткове, 34.5 – число з плаваючою точкою, 0b1011 – ціле двійкове, 0o377 – вісімкове, 0xFF – шістнадцяткове, `Infinity` – $+\infty$, `NaN` – помилкове число);

– логічний (`boolean`) – визначається відсутністю лапок і використовується для значень `true` = 1 або `false` = 0;

– символьний (`Symbol`) – тип унікального незмінного ідентифікатору;

– невизначений (`undefined`) – тип будь-якої неініціалізованої змінної (такої, якій не було надане значення).

4. Спеціальні типи даних:

– порожній (`null`) – відсутність даних у оголошеній змінній;

– об'єкт (`object`) – програмний об'єкт (посилання на нього);

– функція (`function`) – визначення функції.

5. Основні оператори:

– додавання як бінарний, перетворення рядка на число як унарний;

– віднімання як бінарний, зміна знаку як унарний;

– множення;

– ділення;

– ділення за модулем (остача);

– інкремент (збільшення на 1);

– надання значення;

– додати та надати значення;

--	відняти та надати значення;
*	помножити та надати значення;
/	поділити та надати значення;
%	знайти остачу від ділення та надати значення;
+	додати та надати значення;
==	дорівнює;
!=	не дорівнює;
>	більше;
>=	більше або рівний;
<	менше;
<=	менше або рівний;
===	ідентичний (дорівнює та одного типу);
!==	не ідентичний;
!	логічне заперечення;
	логічна диз'юнкція;
&&	логічна кон'юнкція;
&	побітова кон'юнкція;
	побітова диз'юнкція;
^	бінарна виключна диз'юнкція;
<<	побітовий зсув вліво;
>>	побітовий зсув вправо (із збереженням знаку);
>>>	побітовий зсув вправо (без збереження знаку);
~	побітове заперечення.

6. Визначення функції:

```
function ім'я_функції(параметр1, параметр2, ..., параметрn)
{
    оператори
    return значення_що_повертається;
}
```

або

```

var ім'я_функції = function(парам1, парам2, ..., парамn)
{
    оператори
    return значення_що_повертається;
}

```

або

```

var ім'я_функції = new Function('парам1', 'парам2', ...,
'парамn', 'оператори; return значення_що_повертається');

```

7. Умовний вираз:

```

if(умова)
{
    оператори1
}
else // інакше, якщо умова не виконалась
{
    оператори2
}

```

або

```

результат = умова ? оператори1 : оператори2;

```

8. Оператор вибору надає можливість порівняти одну змінну з великою кількістю констант. Наприклад:

```

var a ;
switch ( a )
{
case 1 : // якщо a = 1
    оператори
    [ break ; ]
case 2 : // якщо a = 2
    оператори
    [ break ; ]
default : // якщо a = 3
    оператори
    [ break ; ]
}

```

case порівнює змінну, зазначену в switch (змінна). break перериває виконання case або default, тобто якщо він буде відсутнім при виконанні хоча б першого case, виконуються всі наступні та default. default виконається тільки якщо не виконається жоден із операторів case.

9. Цикли:

– while – цикл з передумовою, який триватиме до того моменту, коли умова не перестане виконуватись:

```
while (умова)
{
    оператори
}
```

– do ... while – цикл з післяумовою, який відрізняється від циклу while тим, що умова перевіряється наприкінці виконання блоку:

```
do {
    оператори
}while (умова)
```

– for – ітераційний цикл:

```
for (var змінна = початкове_значення; умова; крок циклу) {
    оператори
}
```

або

```
for (var ім'я_властивості in деякий_об'єкт) {
    // дії за допомогою деякий_об'єкт[ім'я_властивості];
}
```

10. Типи JavaScript поділяються на примітивні та об'єктні. Об'єкти можуть розглядатися як асоціативні масиви або хеші, тому часто реалізуються з використанням цих структур даних. *Стандартні об'єкти*: Array, Date, Error, Math, Boolean, Function, Number, Object, RegExp, String. Інші об'єкти – це «хост-об'єкти», що визначаються не мовою, а середовищем виконання (наприклад, у браузері типові хост-об'єкти належать до DOM).

Об'єкти можуть бути створені за допомогою конструктора або літерала об'єкта (останнє є основою об'єктної нотації JavaScript – JSON):

```
var anObject = new Object(); // конструктор
// літерали
var objectA = {};
var objectB = {index1:'значення 1', index2:'значення 2'};
```

Як було показано вище, для доступу до даних та методів об'єкту застосовується оператор «точка» (.).

Все, що стосується синтаксису JavaScript/ECMAScript, можна знайти у багатьох джерелах – наприклад, якісних відеолекціях Дугласа Крокфорда (<https://youtu.be/playlist?list=PLEzQf147-uEpvTa1bHDNlxUL2klHUMHJu>).

A-Frame використовує ECS (Entity – Component – System) – шаблон проектування комп'ютерних ігор, основними поняттями якого є Entity (сутність), Component (компонент) та System (система). Сутність – це контейнер для компонентів. Сутності є основою всіх об'єктів на сцені, але без компонентів сутності нічого не роблять і не надають. Компонент – це невеликий об'єкт, який реалізує певну структуру даних та відповідає за окрему частину логіки роботи програми. Кожен тип компонента можна прикріпити до сутності, щоб надати їй певної властивості. Системи управляють набором сутностей, об'єднаних деякими компонентами. Вони не є обов'язковими.

У A-Frame цей шаблон проектування реалізований за допомогою атрибутів. Як сутностей використовуються будь-які примітиви A-Frame – `a-scene`, `a-box`, `a-sphere` та ін. Але особливе місце займає `a-entity`, ім'я говорить саме за себе. Всі інші примітиви є по суті обгортками для компонентів і зроблені для зручності, тому що будь-який елемент можна створити і за допомогою `a-entity`. Наприклад, примітив `a-box` можна реалізувати у такий спосіб:

```
<a-entity
  geometry="primitive: box; width: 1; height: 1; depth: 1">
</a-entity>
```

`geometry` у даному випадку є компонентом, який був доданий до сутності `<a-entity>`. Сам по собі `<a-entity>` не має будь-якої логіки (в глобальному сенсі), а компонент `geometry` по суті перетворює його на куб або

що-небудь інше. Іншим, не менш важливим, ніж `geometry`, компонентом є `material`. Він додає до геометрії матеріал. Матеріал відповідає за те, чи буде наш куб блищати як метал, чи буде мати будь-які текстури та ін.

Будь-компонент у A-Frame повинен бути зареєстрований глобально через спеціальну конструкцію:

```
AFRAME.registerComponent('hello-world', {
  init: function () {
    console.log('Hello, World!');
  }
});
```

Створений компонент можна додати на сцену, як і будь-який інший елемент:

```
<a-entity hello-world></a-entity>
```

Створення компоненту приводить до виклику методу `init`, що виводить повідомлення до консолі веб-браузера. У компоненті можуть бути визначені також методи:

`update` – викликається при ініціалізації разом з `init` та при оновленні будь-якого властивості компонента;

`remove` – викликається після видалення компонента або сутності, що його містить;

`tick` – викликається кожного разу перед відображенням чи оновленням (рендерингом) сцени;

`tock` – викликається після рендерингу сцени;

`play` – викликається кожного при поновленні рендерингу сцени;

`pause` – викликається при зупинці рендерингу сцени;

`updateSchema` – викликається кожен раз після оновлення схеми.

Схема описує властивості компонента та визначається у такий спосіб:

```
AFRAME.registerComponent('my-component', {
  schema: {
    arrayProperty: {type: 'array', default: []},
    integerProperty: {type: 'int', default: 5}
```

```

    }
  }

```

У даному випадку компонент `my-component` буде містити дві властивості – `arrayProperty` та `integerProperty`. Щоб передати їх до компонента, потрібно задати значення відповідного атрибута.

```

<a-entity          my-component="arrayProperty:          1,2,3;
integerProperty: 7"></a-entity>

```

Отримати ці властивості всередині компонента можна через властивість `data`. Отримати властивість `data` компонента із сутності, до якої він доданий, можна за допомогою `getAttribute`, а за допомогою `setAttribute` – встановити властивість у певне значення.

Системи у `A-Frame` задаються параметрами `a-scene` та реєструються через `AFRAME.registerSystem (name, definition)` – саме так бібліотекою `Ar.js` зареєстрована система `arjs`. На відміну від компонентів, системи надають лише методи `init`, `play`, `pause`, `tick` та `tock`.

`A-Frame` активно використовує об'єктну модель веб-браузера:

а) доступ до будь-якого об'єкту `A-Frame` може бути отриманий через `document.querySelector`, `document.getElementById` тощо;

б) різні компоненти можуть обмінюватись повідомленнями, для чого один з них повинен генерувати повідомлення за допомогою функцією `emit` (приймає три параметри: назву події; дані, які треба передати; ознака спливання (`bubbling`) події), а інший – обробляти («прослуховуючи» чергу події) за допомогою методу, визначеного у `addEventListener`;

в) до об'єктів `A-Frame` можуть бути застосовані методи `setAttribute` (надання атрибуту значення), `removeAttribute` (видалення атрибуту), `createElement` (створення елемента) та `removeChild` (видалення).

Постійно поповнюваний перелік нових компонентів `A-Frame`, розроблених користувачами та оформлених у пакети `npm`, доступний за посиланням <https://www.npmjs.com/search?q=aframe-component>

Для того, щоб застосувати компонент із списку, необхідно традиційно

перейти до його репозитарію, завантажити та підключити відповідний файл. Інший спосіб – скористатись сервісом unpkg, який надає можливість швидко завантажити будь-який файл з npm-паketу за посиланням вигляду

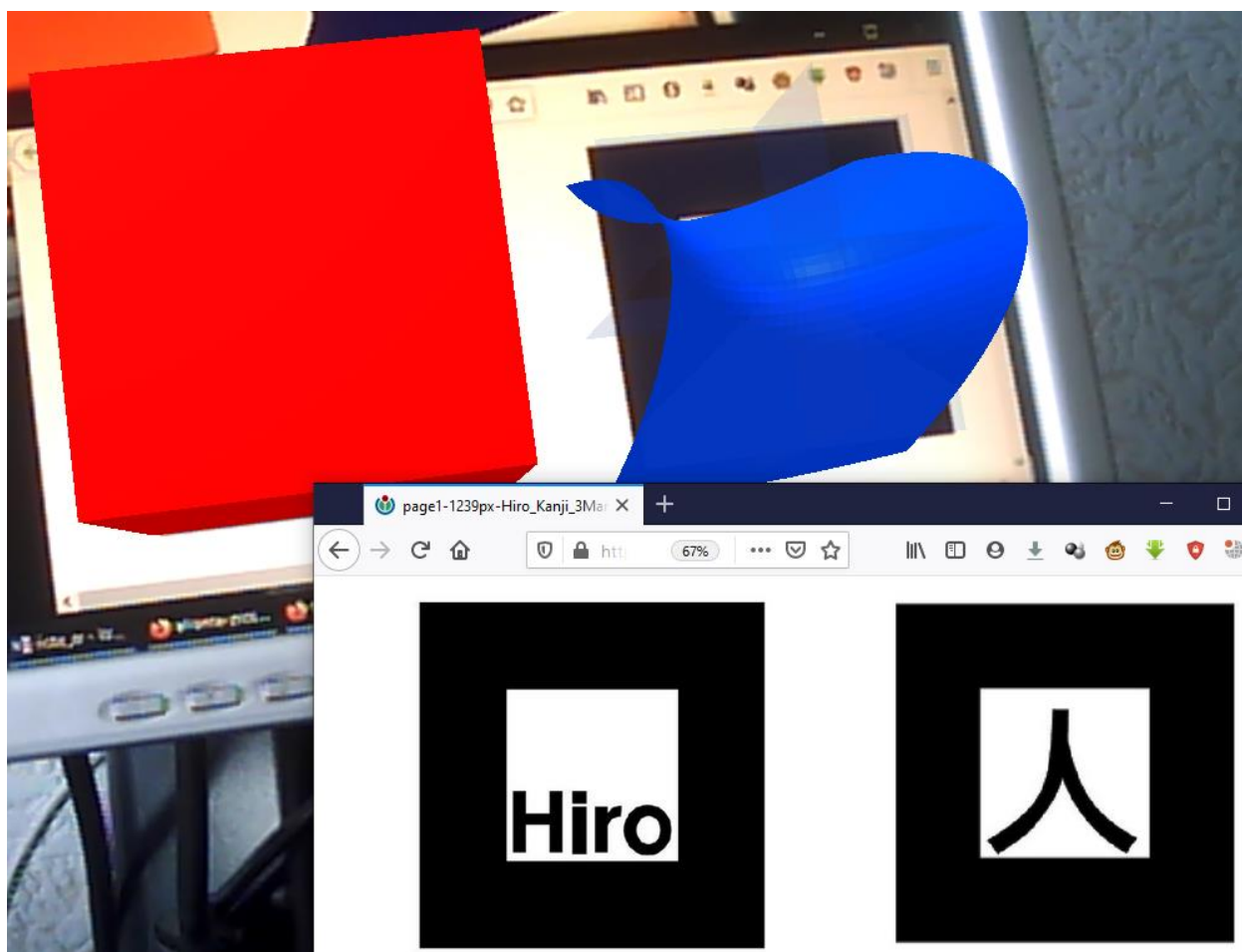
`unpkg.com/:package@:version/:file`

В якості прикладу скористаємось компонентом для побудови 3D-поверхонь:

```
<script src="https://unpkg.com/aframe-plot-component/dist/aframe-plot-component.min.js"> </script>
```

Для його використання замінимо блакитний куб, що пов'язаний із другим маркером, на поверхню:

```
<a-entity plot="function: ((3*x)^2 - (4*y)^2)/4;
  order: 32;
  show_zero_planes: true;
  bounds: -0.5 0.5 -0.5 0.5 -0.5 0.5;
  color: #04F"> </a-entity>
```

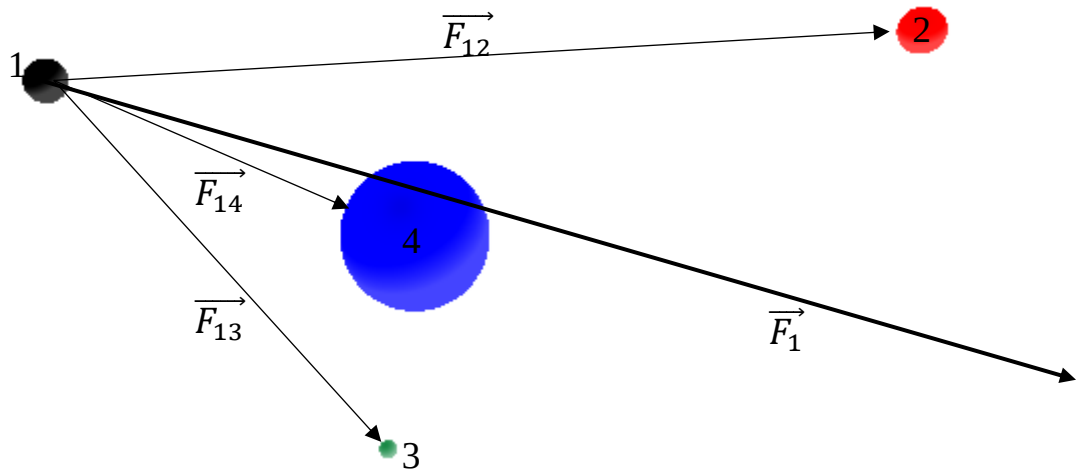


2.7. Приклад: модель Сонячної системи

Надамо об'єктам A-Frame фізичного змісту, поставивши задачу моделювання руху тіл в полі сил тяжіння. Для цього згадаємо два основних закони руху:

- 1) під дією сили $\vec{F}$ тіло масою m набуває прискорення $\vec{a}$;
- 2) сила F , з якою притягуються два тіла, прямо пропорційна добутку їх мас m_1 та m_2 і обернено пропорційна квадрату відстані r між ними.

У першому випадку традиційно використовується векторний запис, у другому – скалярний. Приведемо запис до єдиної форми, вважаючи, що тіла рухаються у тривимірному просторі.



Прискорення $\vec{a}_1$ набуває тіло масою m_1 під дією сили $\vec{F}_1$, яка є сумою сил $\vec{F}_{12}$, $\vec{F}_{13}$ та $\vec{F}_{14}$. Кожна з них, у свою чергу, за законом всесвітнього тяжіння визначається формулою виду

$$F_{ij} = G \frac{m_i m_j}{r_{ij}^2}.$$

У загальному випадку сила $\vec{F}_{ij}$ напрямлена від тіла масою m_i до тіла масою m_j уздовж вектору $\vec{r}_{ij} = \vec{r}_j - \vec{r}_i$, що з'єднує центри їх мас ($\vec{r}_i$ – координатний радіус-вектор тіла i), тому попередню формулу краще переписати у такий спосіб:

$$\vec{F}_{ij} = G \frac{m_i m_j}{|\vec{r}_{ij}|^2} \cdot \frac{\vec{r}_{ij}}{|\vec{r}_{ij}|} = G \frac{m_i m_j}{|\vec{r}_{ij}|^3} \vec{r}_{ij}.$$

Тоді сила $\vec{F}_i = m_i \vec{a}_i$, що діє на тіло масою m_i з боку інших n тіл, може бути записана як їх сума:

$$\vec{F}_i = \sum_{\substack{j=1 \\ (i \neq j)}}^n \vec{F}_{ij} = \sum_{\substack{j=1 \\ (i \neq j)}}^n G \frac{m_i m_j}{|\vec{r}_{ij}|^3} \vec{r}_{ij} = G m_i \sum_{\substack{j=1 \\ (i \neq j)}}^n \frac{m_j \vec{r}_{ij}}{|\vec{r}_{ij}|^3},$$

або

$$m_i \vec{a}_i = G m_i \sum_{\substack{j=1 \\ (i \neq j)}}^n \frac{m_j \vec{r}_{ij}}{|\vec{r}_{ij}|^3},$$

звідки визначаємо прискорення $\vec{a}_i$, що набуває тіло i під дією інших:

$$\vec{a}_i = G \sum_{\substack{j=1 \\ (i \neq j)}}^n \frac{m_j \vec{r}_{ij}}{|\vec{r}_{ij}|^3}.$$

Ураховуючи, що $\vec{a}_i = \frac{d\vec{v}_i}{dt}$, а $\vec{v}_i = \frac{d\vec{r}_i}{dt}$, після переходу до скінченних різниць отримуємо наступну розрахункову схему:

- 1) визначити прискорення, яке набуває тіло i під дією з боку інших тіл;
- 2) визначити зміну швидкості i -го тіла: $\Delta \vec{v}_i = \vec{a}_i \Delta t$;
- 3) визначити зміну координати i -го тіла: $\Delta \vec{r}_i = \vec{v}_i \Delta t$.

Її застосування потребує визначення початкових координат та швидкостей. Для цього скористаємось припущення про те, що усі тіла утворюють «парад планет», розташовуючись на вісі x та рухаючись вздовж перпендикулярної до неї вісі y .



Тоді початкова швидкість всіх тіл, за винятком першого (центрального), може бути визначена як

$$v_y = \frac{2\pi R}{T},$$

де R – середня відстань планети від Сонця, а T – період її обертання навколо Сонця.

Для зручності роботи спочатку налагодимо модель у віртуальній реальності на невеликій кількості об'єктів, узявши планети земної групи, а після того, як вона стане працездатною, додамо інші планети та перенесемо у доповнену реальність.

Після підключення бібліотеки A-Frame зареєструємо новий компонент – planet:

```
const day = 24.0*60*60; //тривалість земного дня у секундах
AFRAME.registerComponent('planet', {
  schema: {
    name: {type: 'string', default: ""}, //ім'я планети
    //середня відстань планети від Сонця
    dist: {type: 'number', default: 0},
    mass: {type: 'number', default: 0}, //маса планети, кг
    T: {type: 'int', default: 0}, //планетарний рік, земних днів
    v: {type: 'array', default: [0,0,0]}, //вектор швидкості
    a: {type: 'array', default: [0,0,0]}, //вектор прискорення
    //координатний радіус-вектор
    pos: {type: 'array', default: [0,0,0]}
  },

  init: function () {
    this.data.T*=day; //переводимо із земних днів у секунди
    this.data.pos[0]=this.data.dist; //розташовуємо на вісі x
    //візуальну позицію виражаємо у мільйонах кілометрів
    this.el.setAttribute('position',this.data.dist/1e9+' 0 0');
    if(this.data.T!=0)//для всіх об'єктів, крім Сонця,
      //обчислюємо початкову швидкість вздовж вісі y
    this.data.v[1] = 2*Math.PI*this.data.dist/this.data.T;
```

```

    }
  });

```

Метод `init` забезпечує правильне відображення об'єкту на сцені та додатково обчислює початкову швидкість об'єкта відносно центрального тіла, адже Сонце у даній моделі теж створюватиметься компонентом `planet`. Для того, щоб відрізнити його від інших об'єктів, встановимо для нього значення параметру `T` у 0.

Новий компонент не визначає спосіб відображення об'єкту – для цього використовується відповідна сутність:

```

<a-scene>
  <a-sky color="black"></a-sky>
  <a-camera position="0 0 300" cursor-visible="true" cursor-
scale="2" cursor-color="#0095DD" cursor-opacity="0.5"></a-camera>
  <a-entity geometry="primitive: sphere; radius: 20.510"
material="color: yellow" planet="dist: 0; mass: 1.989e30; name:
Sun"></a-entity>
  <a-entity geometry="primitive: sphere; radius: 2.4397"
material="color: #AA5588" planet="dist: 57.910e9; mass: 3.285e23;
T: 88; name: Mercury"></a-entity>
  <a-entity geometry="primitive: sphere; radius: 6.0518"
material="color: white" planet="dist: 108.2e9; mass: 4.876e24; T:
224.7; name: Venus"></a-entity>
  <a-entity geometry="primitive: sphere; radius: 6.371"
material="color: cyan" planet="dist: 149.6e9; mass: 6e24; T: 365;
name: Earth"></a-entity>
  <a-entity main></a-entity>
</a-scene>

```

У створених сутностях можна легко змінити геометрію та матеріал об'єкту, і це не порушить реалізовану в компоненті логіку роботи. Зверніть увагу на розташування камери: вона відсунута по вісі `z` так, щоб можна було побачити орбіту поки що найвіддаленішої планети (подвоєна середня відстань Землі від Сонця) – при додаванні нових планет камеру доведеться відповідно відсувати далі.

За інтегрування рівнянь руху відповідатиме компонент main:

```
AFRAME.registerComponent('main', {
  init: function() {
    this.solar_system = document.querySelectorAll('[planet]');
  },

  tick: function (time, deltaTime) {
    const dt = day/3; //крок інтегрування
    const G=6.67e-11; //гравітаційна стала
    for(var i = 0; i<this.solar_system.length; i++) {
      planet_i=this.solar_system[i].getAttribute('planet');
      planet_i.a[0]=planet_i.a[1]=planet_i.a[2]=0;

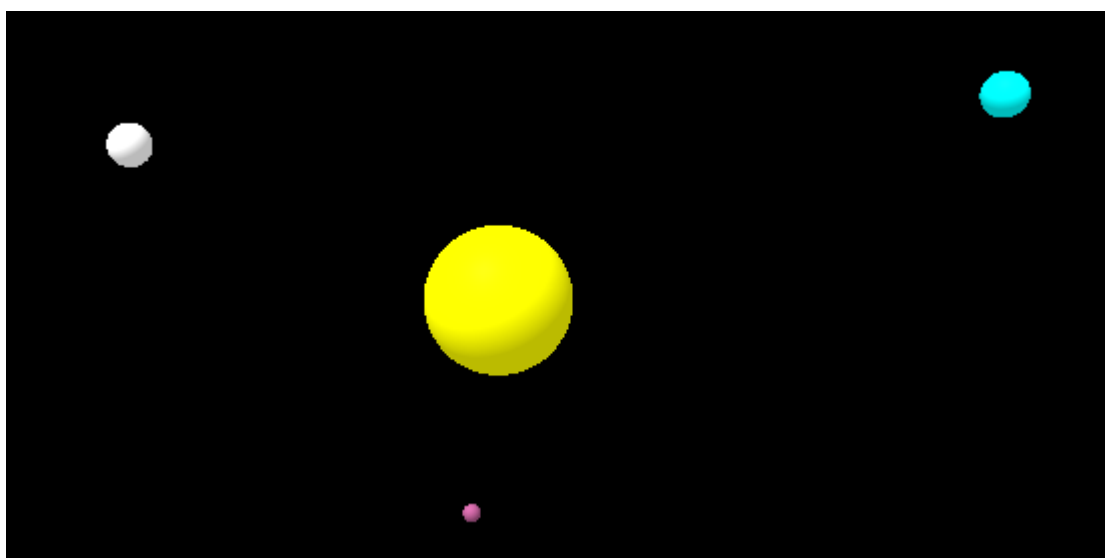
      for(var j = 0; j<this.solar_system.length; j++) {
        planet_j=this.solar_system[j].getAttribute('planet');
        if(i!=j) {
          deltapos = [0,0,0];
          for(var k = 0; k < 3; k++)
            deltapos[k]=planet_j.pos[k]-planet_i.pos[k];
          var r=Math.sqrt(Math.pow(deltapos[0],2)+
            Math.pow(deltapos[1],2)+Math.pow(deltapos[2],2));
          for(var k = 0; k < 3; k++)
            planet_i.a[k]+=G*planet_j.mass*deltapos[k]/
              Math.pow(r, 3);
        }
      }
      for(var k = 0; k < 3; k++)
        planet_i.v[k]+=planet_i.a[k]*dt;
      for(var k = 0; k < 3; k++)
        planet_i.pos[k]+=planet_i.v[k]*dt;
      this.solar_system[i].setAttribute('position',
        (planet_i.pos[0]/1e9)+' '(planet_i.pos[1]/1e9)+
        ' '(planet_i.pos[2]/1e9));
    }
  }
});
```

```
});
```

Для того, щоб отримати всі визначені засобами A-Frame сутності з компонентом `planet`, у методі `init` компоненту `main` використовується метод `document.querySelectorAll`: отриманий масив об'єктів зберігається у змінній `solar_system`. Розмірність масиву й визначає кількість об'єктів гравітаційної взаємодії n . Кожен об'єкт масиву `solar_system` містить велику кількість елементів – для полегшення доступу до необхідних властивостей створених компонентів викликом `getAttribute` створюються дві змінні – `planet_i` та `planet_j`: просто запис із ними буде коротшим, ніж звернення до елементів `solar_system`. Ці та інші повторювані дії розміщені у методі `tick`, що відповідає за опрацювання події від таймеру.

Перш ніж розпочати обчислення прискорення, відповідні координатам x , y та z елементи масиву `a[0]`, `a[1]` та `a[2]` встановлюються у 0 (більш короткий спосіб показаний при визначенні `deltapos`). `deltapos` відповідає вектору $\vec{r}_{ij}$ (відповідно `planet_j.pos - r_j`, а `planet_i.pos - r_i`), а r – його модулю. Цикли за змінною k проходять координати всіх векторів.

Обчислені координати об'єкту `pos` є масивом компоненту `planet`, у той час як атрибут `position` – рядком із `geometry`, що вимагає відповідного перетворення.



Додавання до сцени інших планет та компоненту `arjs` залишимо як завдання для самостійної роботи.

2.8. Застосування маркерів

Тег `<a-marker>` має такі основні атрибути:

`type` – тип маркеру: `'pattern'`, `'barcode'`, `'unknown'`;

`size` – розмір маркеру в метрах;

`url` – посилання на шаблон маркеру, якщо його тип – `'pattern'`;

`value` – значення коду, якщо тип маркеру – `'barcode'`;

`preset` – вибір стандартного маркеру (`'hiro'`, `'kanji'`);

`emitevents` – генерування подій `'markerFound'` та `'markerLost'`, якщо встановлений у `'true'`;

`smooth` – вмикає/вимикає згладжування зображення з камери (за замовчанням – `'false'`).

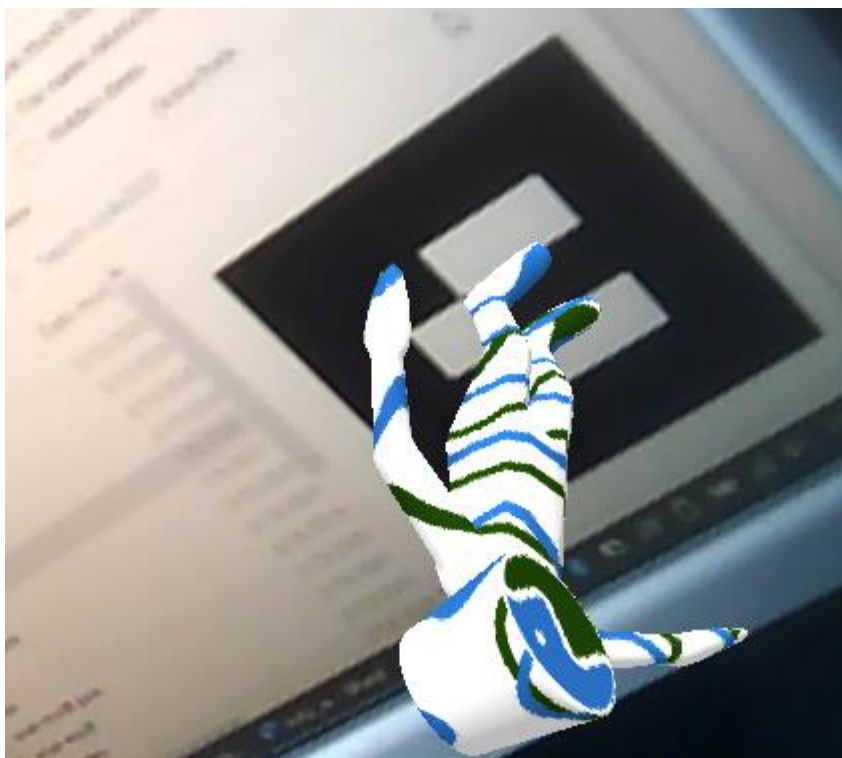
Тип `'barcode'` відповідає двовимірному (матричному) коду ARToolKit. У репозитарії поточного розробника AR.js Ніколо Капріньолі за посиланням <https://github.com/nicolocarpignoli/artoolkit-barcode-markers-collection> можна знайти зображення для всіх типів матричних кодів, що задаються атрибутом `matrixCodeType` параметру `arjs` примітиву `a-scene`. Чим простіше матричний код, тим легше його розпізнати, але й тим менше маркерів можна створити за його допомогою: `3x3_HAMMING63` підтримує 8 маркерів (відстань Хеммінга – 3), `3x3_PARITY65-32` (1), `4x4_BCH_13_5_5-32` (5), `4x4_BCH_13_9_3-512` (3), `5x5_BCH_22_7_7-128` (7), `5x5_BCH_22_12_5-4096` (5). Чим більше відстань Хеммінга, тим краще маркер розпізнаватиметься.

Приклад налаштування сцени для використання матричних кодів:

```
<a-scene      vr-mode-ui="enabled:      false"      embedded
arjs="debugUIEnabled:false;      detectionMode:      mono_and_matrix;
matrixCodeType: 3x3;">
```

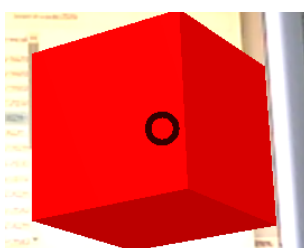
Відключення компонента `vr-mode-ui` надає можливість прибрати стандартні іконки A-Frame для переходу до повноекранних VR/AR режимів. Сам маркер створюється так:

```
<a-marker type="barcode" value='7>' ... </a-marker>
```



Для опрацювання подій від клавіатури та миші необхідно вказати відповідний обробник події та підключити курсор до камери. У наступному прикладі показується, як можна анімувати об'єкт за подіями від миші, курсор якої має форму чорного кільця:

```
<a-scene vr-mode-ui="enabled: false" embedded arjs='sourceType:
webcam; debugUIEnabled: false; detectionMode: mono_and_matrix;
matrixCodeType: 3x3;'>
  <a-marker type='barcode' value='6'>
    <a-entity geometry="primitive: box" material="color: red"
animation__mouseenter="property: rotation; from:0 0 0; to: 0 0
360; startEvents: mouseenter; dur: 1000"; animation__mouseleave=
"property: components.material.material.color; type: color; from:
blue; to: red; startEvents: mouseleave; dur: 1000";> </a-entity>
  </a-marker>
  <a-entity camera><a-cursor></a-cursor></a-entity>
</a-scene>
```



Інший спосіб – явне використання обробників подій через виклик `addEventListener`. Наприклад, на рівні вікна веб-браузера можна визначити параметри підключеної камери під час її ініціалізації або подію припинення відеопотоку з камери:

```
window.addEventListener('camera-init', (data) => {
    console.log('camera-init', data);
})
window.addEventListener('camera-error', (error) => {
    console.log('camera-error', error);
})
```

Якщо необхідно опрацьовувати події, пов'язані із певною сутністю A-Frame, доцільно увести до неї новий компонент та зареєструвати його. Наприклад, компонент `registerevents`, доданий до маркера, допоможе розібратись, коли він потрапляє до поля зору камери (розпізнається), а коли – виходить з поля зору («губиться»):

```
AFRAME.registerComponent('registerevents', {
  init: function () {
    var marker = this.el;
    marker.addEventListener('markerFound', function() {
      console.log('markerFound', marker.id); });
    marker.addEventListener('markerLost', function() {
      console.log('markerLost', marker.id); });
  }
});
```

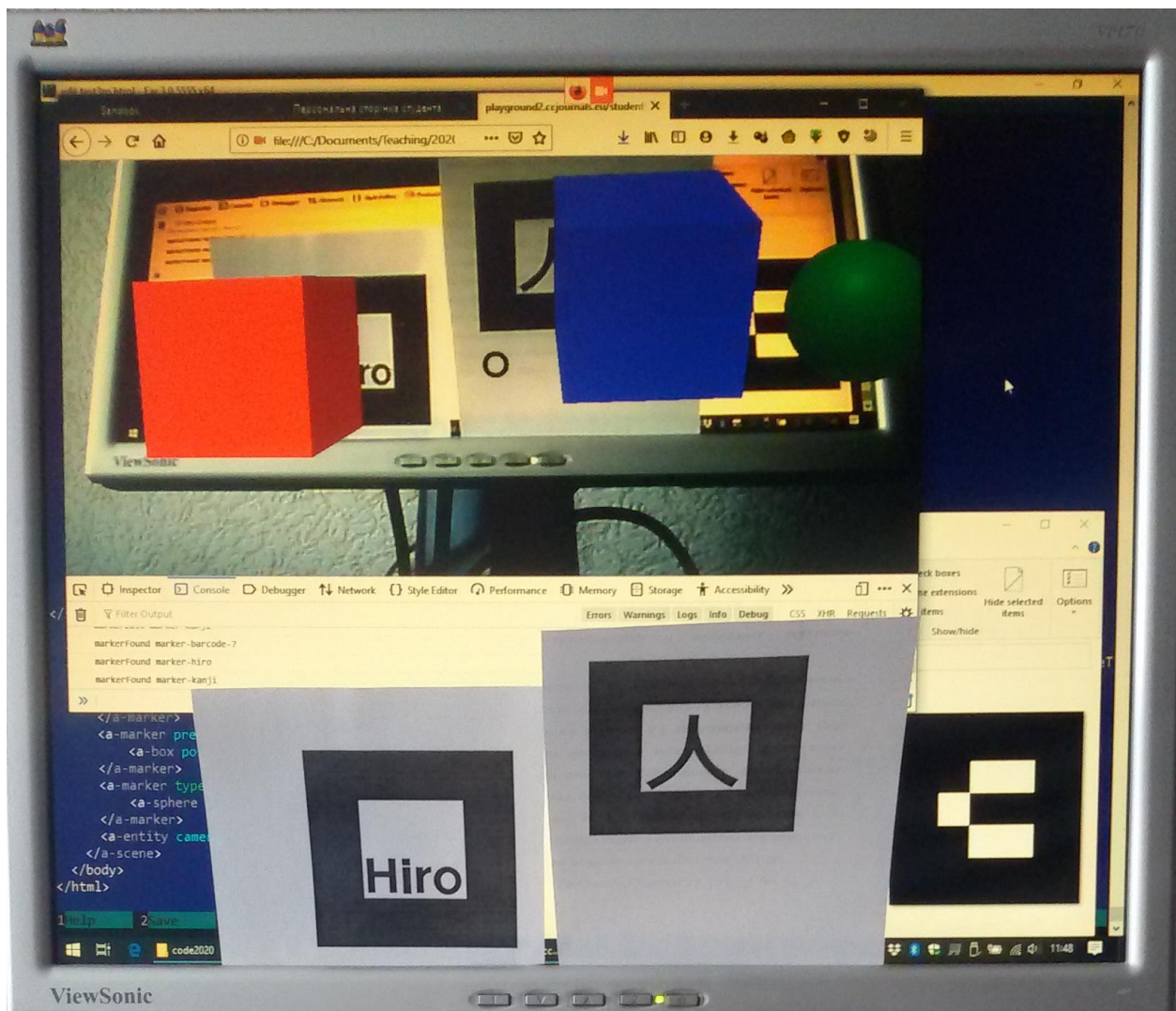
Приклад компонента для видимої сутності, що опрацьовує натискання кнопки миші на ньому:

```
AFRAME.registerComponent('jump', {
  init: function () {
    var obj = this.el;
    obj.addEventListener('click', function () {
      this.setAttribute('position', (Math.random()-0.5)+ " 0.5 0");
    });
  }
});
```

```
});
```

Приклад сцени з трьома незалежними маркерами, об'єкти яких реагують на натискання миші переміщенням:

```
<a-marker preset="hiro" id='marker-hiro' registerevents>
  <a-box position='0 0.5 0' material='color: red;' jump></a-box>
</a-marker>
<a-marker preset="kanji" id='marker-kanji' registerevents>
  <a-box position='0 0.5 0' material='color:blue;' jump></a-box>
</a-marker>
<a-marker type='barcode' value='6' id='marker-barcode-7'
  registerevents>
  <a-sphere position='0 0.5 0' radius="0.5"
    material='color: green;' jump></a-sphere>
</a-marker>
<a-entity camera><a-cursor></a-cursor></a-entity>
```



AR.js надає можливість використання усіх стандартних маркерів, що підтримує ARToolKit. Крім вже використаних 'hiro' та 'kanji', це 'letterA', 'letterB', 'letterC', 'letterD', 'letterF', 'letterG' та ряд інших. Частина маркеру, що детектується – чорний квадрат зі стороною 1, усередині якого розміщено зображення. Опис розміщується у текстовому файлі (як правило, його ім'я звершується на .patt) – саме він використовується при розпізнаванні, а не звичне зображення.

```

patt.kanji - Notepad
File Edit Format View Help
214 225 240 225 214 240 216 204 214 227 181 192 198 192 181 192
240 240 240 240 240 240 225 232 240 240 240 240 240 240 240 236
240 240 240 240 240 240 75 128 220 240 240 240 240 240 240 240
240 240 240 240 240 240 106 53 240 240 240 240 240 240 240 237
240 240 240 240 240 238 118 31 240 240 240 240 240 240 240 234
240 240 240 240 240 74 49 207 240 240 240 240 240 240 240 240
240 240 240 240 240 53 54 177 240 240 240 240 240 240 240 240
240 240 240 240 240 64 31 130 240 240 240 240 240 240 240 219
240 240 240 240 240 180 37 57 78 228 240 240 240 240 240 240
240 240 240 240 240 118 62 157 36 185 240 240 240 240 240 231
240 240 240 240 240 82 65 225 67 80 230 240 240 240 240 217
240 240 240 225 53 76 225 240 156 62 158 240 240 240 240 226
240 240 199 61 9 111 235 240 240 104 58 174 228 240 240 240
240 142 64 26 92 227 240 240 240 229 93 64 170 226 238 216
90 26 12 156 240 240 240 240 240 240 204 95 30 117 192 200
156 16 195 233 235 240 236 240 238 239 240 186 93 53 120 237
214 226 240 225 212 240 216 204 212 226 181 192 198 192 185 194
Ln 28, Col 58 100% Unix (LF) UTF-8

```



Усього файл шаблону маркера зберігає 12 його зображень у 4 орієнтаціях. Створення маркеру, що гарно розпізнається – непроста задача: яким би не був розмір зображення, для використання ARToolKit воно буде закодоване усього 256 елементами (16 рядків на 16 стовпців), тому надмірно деталізоване зображення буде неминуче спрощене. Приклад гарного маркеру:



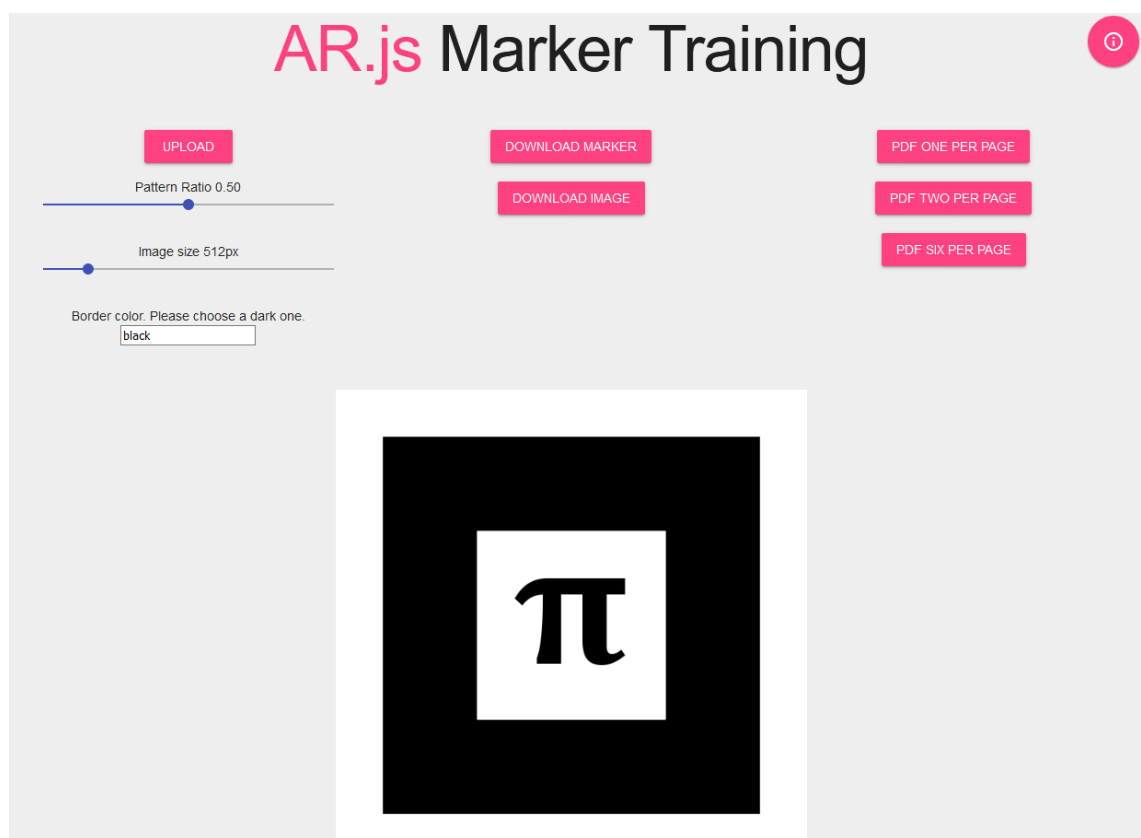
та його кодування (перше зображення):

```

255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 242  95  11   0   0   0   0   0   0 114 255 255 255
255 255 255 101  72  82   0 141 149   0   1 159 202 255 255 255
255 255 255 232 253 141   0 226 239   0   2 255 255 255 255 255
255 255 255 255 255 135   0 226 239   0   2 255 255 255 255 255
255 255 255 255 255 119   0 226 239   0   2 255 255 255 255 255
255 255 255 255 255  90   0 226 241   0   2 254 255 255 255 255
255 255 255 255 255  38   0 226 254  37   0  90 182 255 255 255
255 255 255 255 255 195 191 247 255 229 164 210 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255
255 255 255 255 255 255 255 255 255 255 255 255 255 255 255 255

```

Простий спосіб створити власний маркер – скористатись генератором маркерів <https://jeromeetienne.github.io/AR.js/three.js/examples/marker-training/examples/generator.html>



Отриманий текстовий файл із описом маркеру можна завантажити разом із його зображенням та використати для розпізнавання:

```
<a-marker type='pattern' url='pattern-pi.patt'>
...
</a-marker>
```

`a-marker` використовує статичну камеру, що розташована на початку координат та напрямлена у від’ємний бік вісі z . Видимість об’єктів, пов’язаних із маркером, встановлюється залежно від того, чи розпізнаний маркер камерою. Якщо один маркер використовується для того, щоб ініціювати певний процес, доцільно скористатись `a-marker-camera`. Після виходу маркеру з поля зору камери створені об’єкти прив’язуються до переміщення камери – саме за такого підходу модель Сонячної системи буде виглядати найкраще.

Для покращення розпізнавання доцільно використовувати асиметричні маркери, наприклад:

Д,Б!

Ще один спосіб покращити розпізнавання – використання друкованих маркерів достатнього розміру. Вирізаючи маркер, залишайте біле поле шириною принаймні в половину чорної рамки – контраст є суттєвим при розпізнаванні. Якщо й це не допомагає, можна збільшити роздільну здатність камери у компоненті `ar.js` – за замовчанням вона 640x480. Наприклад, для FullHD камери можна вказати `arjs="sourceType: webcam; sourceWidth:1280; sourceHeight:960; displayWidth: 1280; displayHeight: 960; "`.

У лютому 2020 року Ніколо Капріньолі анонсовано створення AR.js Studio, а також підтримка NFT (Natural Feature Tracking) – використання довільних зображень в якості маркерів (<https://carnaux.github.io/NFT-Marker-Creator/>) – у третій версії AR.js.

З РОЗРОБКА ЗА ДОПОМОГОЮ THREE.JS

3.1. Приклад: побудова трикутника

A-Frame рекомендує не розміщувати «голий» JavaScript код, а прив'язувати його до компонентів задля дотримання архітектури ECS – саме так було зроблено у моделі Сонячної системи. Проте, проектуючи нові компоненти, зовсім не обов'язково застосовувати виключно A-Frame – можна послуговуватись і тим, на чому він базується, аж до рівня WebGL. Розглянемо це на прикладі створення AR-програми, що передбачає одночасне використання кількох маркерів – побудову трикутника. Для цього розмістимо на сцені три сфери, прив'язані до маркерів із зображенням літерам А, В та С – шаблони для їх розпізнавання входять до складу AR.js:

```
<a-scene embedded vr-mode-ui="enabled: false;"
arjs="debugUIEnabled: false; detectionMode: mono_and_matrix;"
  <a-marker type="pattern"
url="https://raw.githubusercontent.com/jeromeetienne/AR.js/master/data/multimarkers/multi-abcdef/patt.a" id="A" registerevents>
    <a-sphere radius="0.10" color="red"></a-sphere>
    <a-entity id="lineAB"></a-entity>
  </a-marker>
  <a-marker type="pattern"
url="https://raw.githubusercontent.com/jeromeetienne/AR.js/master/data/multimarkers/multi-abcdef/patt.b" id="B" registerevents>
    <a-sphere radius="0.10" color="red"></a-sphere>
    <a-entity id="lineBC"></a-entity>
  </a-marker>
  <a-marker type="pattern"
url="https://raw.githubusercontent.com/jeromeetienne/AR.js/master/data/multimarkers/multi-abcdef/patt.c" id="C" registerevents>
    <a-sphere radius="0.10" color="red"></a-sphere>
    <a-entity id="lineAC"></a-entity>
  </a-marker>
  <a-entity camera></a-entity>
```

```

    <a-entity run></a-entity>
</a-scene>

```

Кожна із сфер є аналогом точки – вершини трикутника – та має ідентифікатор "A", "B" та "C" відповідно. Крім сфери, до маркеру прив'язані сутності без зовнішнього відображення – вони резервуватимуть місце для циліндрів – сторін трикутника – що з'єднуюватимуть сфери. Сутність із компонентом `run` відповідатиме за логіку роботи програми, а компонент `registerevents` міститиме обробники подій «знаходження» та «втрати» маркера:

```

let markerVisible = { A: false, B: false, C: false };
AFRAME.registerComponent('registerevents', {
  init: function () {
    let marker = this.el;
    marker.addEventListener('markerFound', function() {
      markerVisible[ marker.id ] = true;
    });
    marker.addEventListener('markerLost', function() {
      markerVisible[ marker.id ] = false;
    });
  }
});

```

`markerVisible` – об'єкт JavaScript, що використовується для зберігання відомостей про те, які маркери в поточний момент є детектовані. Компонент `run` складають дві функції – початкова `init` та періодична `tick`:

```

AFRAME.registerComponent('run', {
  init: function() {
    this.A = document.querySelector("#A");
    this.B = document.querySelector("#B");
    this.C = document.querySelector("#C");
    this.p0 = new THREE.Vector3();
    this.p1 = new THREE.Vector3();
    this.p2 = new THREE.Vector3();
    let material = new THREE.MeshLambertMaterial(
{color:0xFF0000});

```

```

    let geometry=new THREE.CylinderGeometry( 0.05, 0.05, 1, 12);
    geometry.applyMatrix( new THREE.Matrix4().makeTranslation(
0, 0.5, 0 ) );
    geometry.applyMatrix( new THREE.Matrix4().makeRotationX(
THREE.Math.degToRad( 90 ) ) );
    this.cylinderAB = new THREE.Mesh( geometry, material );
    this.lineAB = document.querySelector('#lineAB').object3D;
    this.lineAB.add( this.cylinderAB );
    this.cylinderAB.visible = false;
    this.cylinderBC = new THREE.Mesh( geometry, material );
    this.lineBC = document.querySelector('#lineBC').object3D;
    this.lineBC.add( this.cylinderBC );
    this.cylinderBC.visible = false;
    this.cylinderAC = new THREE.Mesh( geometry, material );
    this.lineAC = document.querySelector('#lineAC').object3D;
    this.lineAC.add( this.cylinderAC );
    this.cylinderAC.visible = false;
},

tick: function (time, deltaTime) {
    if ( markerVisible["A"] && markerVisible["B"] ) {
        this.A.object3D.getWorldPosition(this.p0);
        this.B.object3D.getWorldPosition(this.p1);
        let distance = this.p0.distanceTo( this.p1 );
        this.lineAB.lookAt( this.p1 );
        this.cylinderAB.scale.set(1,1,distance);
        this.cylinderAB.visible = true;
    }
    if ( markerVisible["B"] && markerVisible["C"] ) {
        this.B.object3D.getWorldPosition(this.p1);
        this.C.object3D.getWorldPosition(this.p2);
        let distance = this.p1.distanceTo( this.p2 );
        this.lineBC.lookAt( this.p2 );
        this.cylinderBC.scale.set(1,1,distance);
        this.cylinderBC.visible = true;
    }
}

```

```

    if ( markerVisible["A"] && markerVisible["C"] ) {
        this.A.object3D.getWorldPosition(this.p0);
        this.C.object3D.getWorldPosition(this.p2);
        let distance = this.p2.distanceTo( this.p0 );
        this.lineAC.lookAt( this.p0 );
        this.cylinderAC.scale.set(1,1,distance);
        this.cylinderAC.visible = true;
    }
    if ( !markerVisible["A"] )
        this.cylinderAB.visible = this.cylinderAC.visible = false;
    if ( !markerVisible["B"] )
        this.cylinderAB.visible = this.cylinderBC.visible = false;
    if ( !markerVisible["C"] )
        this.cylinderAC.visible = this.cylinderBC.visible = false;
}
});

```

На початку функції `init` у компоненті зберігаються посилання на вершини трикутника – сфери `this.A`, `this.B` та `this.C` – і засобами бібліотеки `Three.js` створюються три точки – координатні вектори `p0`, `p1` та `p2`. Далі засобами тієї ж бібліотеки створюється циліндричні об’єкти, що прикріплюються до раніше створених сутностей `lineAB`, `lineBC` та `lineAC`.

Функція `tick` регулярно аналізує, чи є маркери ідентифіковані попарно та поодиночі. Для того, щоб побудувати трикутник, необхідно побудувати його сторони – відрізки. Відповідно, для побудови відрізка необхідні принаймні дві точки, що задають його початок та кінець: після визначення координат цих точок розраховується відстань між ними, далі циліндричний об’єкт спрямовується з до кінцевої точки і подовжується до неї. Коли маркер точки «зникає», пов’язані з нею відрізки приховуються встановленням атрибуту `visible` у `false`.

Наведений код використовує ряд об’єктів та методів з бібліотеки, яка була використана при розробці `A-Frame – Three.js`. Розглянемо більш детально, як можна розширити функціональність наших AR-програм з її використанням.

